Opti-Q: A Constraint-Based Optimization Framework for Multi-LLM Question Planning

Aamir Hamid[1], Bharg Barot[1], Satvik Racharla[1], Tim Finin[1], Primal Pappachan[2], and Roberto Yus[1]
[1]University of Maryland, Baltimore County, USA. [2]Portland State University, USA.
{ahamid2, bhargvb1, yv04378, finin, ryus}@umbc.edu, primal@pdx.edu

***Abstract*—While large language models (LLMs) enable strong question answering (QA), budgeted deployment is complicated by nondeterminism and heterogeneous resource profiles (cost, latency, and energy). We present OPTI-Q, a database-inspired, cost-based optimizer that implements a *plan-before-execute* paradigm for multi-LLM orchestration. OPTI-Q models LLM invocations as physical operators in an execution DAG and, for each question, searches for plans that optimize answer quality (QoA) while trading off financial cost, latency, and energy under user-specified resource constraints. Plans can include sequential operators that pass intermediate answers as context and parallel/blend operators that run models concurrently and merge their outputs. To search this space without executing each candidate plan, OPTI-Q uses PERFDB, a statistics catalog populated and refreshed from benchmarks and execution traces, to estimate the QoA and resource costs of both individual operators and composed subplans. Using these estimates, OPTI-Q performs Pareto-frontier search and selects a final plan based on user preferences. On MMLU-Pro and SimpleQA under user-specified budgets, OPTI-Q improves average QoA by $\approx 58\%$ and $\approx 41\%$ over baselines at comparable cost, demonstrating that database-style planning yields better quality–resource trade-offs for multi-LLM QA.**

***Index Terms*—Large language models, multi-LLM systems, query planning, multi-objective optimization.**

I. INTRODUCTION

Large Language Models (LLMs) such as GPT-5 [1], Llama 3 [2], and DeepSeek-R1 [3] have transformed natural language processing (NLP) [4], driving advances in question-answering (QA) [5], text summarization [6] and domain-specific reasoning [7]. Their success has fueled adoption across industries, from customer service to healthcare [8], [9]. However, real-world deployment remains challenging due to nondeterminism, leading to hallucinations [10], high computational costs [11], substantial energy consumption [12], and latency trade-offs [13]. Considerable effort has focused on these issues, including increasing accuracy and reliability through advanced prompting strategies (e.g., Chain-of-Thought prompting) or extensive fine-tuning [14]. However, a fundamental challenge persists: different LLMs exhibit varying performance across question/task types [15]. Moreover, recent evidence suggests that a single high-capability LLM is not always optimal; coordinated multi-LLM collaboration can outperform the single "best" model on real-world tasks [16].

Inspired by machine learning (ML) ensembles [17], recent work explores combining multiple LLMs to improve reliability and answer quality [18], [19], as cross-model validation helps mitigate hallucinations and raise response quality [20]. However, naive execution strategies (e.g., always querying all models and combining (*blending*) [21] their outputs, or always choosing a fixed "strong" subset of models) introduce new challenges: higher financial costs and latency, inefficient resource use, and sometimes lower answer quality. Moreover, while many systems rely on dynamic sequential orchestration, where later model choices depend on earlier model outputs (e.g., via LangChain-style chains) [22] or execution-time optimization for cost/throughput [23]. Such decisions can be myopic when made without planning for downstream consequences. For example, in our experiments, a history question was best answered by a plan using Gemma-3:27B, Qwen-2.5:14B, and Phi-4:14B (outperforming other plans by a 1.24x to 1.48× factor), whereas a sports question was best handled by a simple parallel execution of Gemma-3:27B and Qwen-2.5:14B. These variations show that the optimal model combination and execution strategy are question-dependent. A robust multi-LLM QA system must therefore determine the best execution plan dynamically for each question, while accounting for end-to-end cost, latency, energy, and quality.

Database systems face a similar problem where the same declarative query can be executed by many alternative physical plans, and the optimizer uses a cost model and statistics to select a plan before execution [24]. In Multi-LLM QA, many alternative workflows may be possible for a user question, each trading off monetary cost, latency, energy, and answer quality. We therefore cast multi-LLM orchestration as a cost-based (and multi-objective) query-planning problem, where the question serves as the query, each LLM invocation is a physical operator, and an end-to-end workflow is a physical plan whose selection is guided by a model "catalog" of historical performance statistics. Unlike classical query optimization, multi-LLM planning must account for stochastic and semantic operator outputs (quality is uncertain and non-monotonic), output-dependent downstream costs (e.g., token lengths and content can amplify later latency/cost/energy), and inherently multi-dimensional objectives (quality–cost–latency–energy) under user constraints. This necessitates new cost estimation models and multi-objective plan selection beyond traditional single-metric cost minimization.

To tackle these challenges, we introduce OPTI-Q, a cost-based optimizer for multi-LLM QA that selects an execution plan *before* issuing any model calls, mirroring how DBMS optimizers choose physical query plans prior to execution. As illustrated in Figure 1, the plans generated by OPTI-Q

involve combinations of parallel and sequential multi-LLM operations. To select among candidate plans, we propose a technique to estimate, before execution, a plan's cost (financial, latency, energy) and benefit (answer quality) from historical performance statistics on LLMs and their combinations across question types. Using these estimates, OPTI-Q identifies plans representing optimal trade-offs among competing objectives under optional user constraints via multi-objective optimization (MOO). Given the potentially large plan space, which depends on the number of LLMs considered and the complexity of the plan structure, OPTI-Q employs a "pluggable optimization engine" that supports diverse planning strategies, including dynamic programming (DP), Hill Climbing, and Non-dominated Sorting Genetic Algorithm (NSGA-II) to explore candidate plans and select one for execution. This enables OPTI-Q to dynamically balance performance and resource efficiency before committing to costly LLM invocations. The main contributions of this work are:

- A cost/benefit formulation of multi-LLM QA planning to select a workflow under budget and resource constraints.
- A statistics-driven optimizer that enumerates and prunes sequential/parallel/hybrid workflows and estimates quality and resource costs prior to execution.
- A system that integrates the optimizer with an execution engine for real-time routing across open-source LLMs.

We evaluate our approach on two QA benchmarks, one open-ended and one multiple-choice, and compare it against four state-of-the-art baselines. In both, OPTI-Q achieves superior QoA-cost efficiency, improving the average QoA by $\approx 58\%$ and $\approx 41\%$ over the strongest budget-aware baseline at matched per-question cost (budget level b=3). We further analyze the impact of LLM diversity, plan complexity, and budget levels on the trade-offs OPTI-Q discovers.

II. RELATED WORK

Recent work at the intersection of data management and LLMs spans two largely orthogonal directions. The first studies how LLMs can assist database query optimization (e.g., improving cardinality estimation [25], cost modeling [26], or physical operator selection [27]). This topic focuses on optimizing database queries over structured data which differs from our setting, where the challenge is to select and compose multiple LLM calls to answer a natural-language question under quality–cost–latency–energy trade-offs. The second direction is more related to our work and treats LLM calls as expensive, probabilistic operators in query-like workflows, asking how to optimize these AI-powered workflows using system and optimizer principles. Orthogonal to this, orchestration frameworks such as LangChain[1] and DSPy [28] support multi-step flows but leave plan structure and physical choices largely developer-scripted, making optimization implicit.

Early approaches to LLM call orchestration adopt *cascading*, routing requests through models of increasing capacity. FrugalGPT [29] typifies this paradigm by learning a budget-aware cascade that reduces cost while maintaining accuracy. However, cascades often impose a largely fixed structure and can accumulate sequential latency; moreover, their intermediate generations can inflate downstream token usage (and thus cost/latency), making greedy "start-cheap" strategies globally suboptimal. Other ensemble methods, such as LLM-Blender [21] and LLM-TOPLA [30], improve quality by fusing multiple outputs, often assuming that additional inference cost is acceptable. Building on these insights, adaptive *question-time* orchestration chooses models conditioned on the input. ThriftLLM [31] learns per-question ensemble selection within a budget, while Shekhar et al. [32] predict quality pre-inference to choose cost-effective models under latency constraints for summarization. These methods align computation with question difficulty, but they typically (i) optimize a single dominant objective and/or (ii) treat model invocations as largely independent decisions rather than as a structured plan with interactions among heterogeneous LLMs.

Complementary recent work formalizes LLM calls as operators in declarative data systems, enabling optimizer-style reasoning over AI pipelines. Galois [33] emphasizes logical optimizations, such as pushing down filters to reduce expensive LLM calls, while frameworks such as PALIMPZEST [23], Abacus [34], and Stretto [35] develop cost-based optimization, constrained objectives, and execution-time trade-offs. OPTI-Q differs from these declarative systems along two key architectural dimensions. First, prior systems primarily optimize data-processing pipelines over records, documents, or multimodal corpora by selecting physical implementations for a given declarative program or workflow. OPTI-Q instead shifts the optimization boundary to online, per-question planning, dynamically generating and selecting sequential, parallel, or hybrid execution graphs tailored to an individual incoming question. Second, while several existing frameworks handle resource-quality trade-offs through scalarized objectives or fixed workflow templates, OPTI-Q treats QoA, cost, latency, and energy as distinct first-class objectives and explicitly explores Pareto trade-offs at question time to support user-specified priorities and resource constraints.

III. OPTI-Q MULTI-LLM PLANNING

This section formalizes user questions, LLMs, and multi-LLM question plans. Then, it postulates the selection of an optimal plan for a given question under QoA, financial, latency, and energy constraints as a multi-objective optimization (MOO) problem. Finally, it sketches our approach. Frequently used notation appears in Table I.

A. Modeling Multi-LLM Planning

Question model. We model each user question as a tuple $Q = (p_t, T, F_{\text{max}}, L_{\text{max}}, E_{\text{max}}, \text{QoA}_{\text{min}}, W)$, where p_t is the textual prompt, explicitly formulated as a question (e.g., "Who are currently the oldest and youngest presidents of an EU country?"), T represents its topic(s) (e.g., "politics"). The parameters F_{max}, L_{max}, E_{max}, and QoA_{min} specify user constraints that represent the maximum allowable

[1]https://github.com/langchain-ai/langchain

TABLE I: Frequently used notation.

Symbol	Definition
Q = (p_t, T, $F_{\max}$, $L_{\max}$, $E_{\max}$, $QoA_{\min}$, W)	User question; its textual content p_t; topic(s) T; cost, latency, energy, and QoA constraints $F_{\max}, L_{\max}, E_{\max}, QoA_{\min}$; and weights per constraint W.
$L = \{L_1, \ldots, L_n\}$	Set of available LLMs.
$Tok_{L_i}(\cdot), T_i^{\text{out}}$	Input and output token lengths associated with model L_i. $Tok_{L_i}(\cdot)$ is the token-counting function using the tokenizer of L_i, while T_i^{out} is the model's average output length.
$L_i = (f_i, l_i, e_i, qoa_i)$	Model descriptor; placeholders for expected financial cost (USD/token), latency (s/token), energy cost (J/token), and QoA estimate used during planning. These quantities are instantiated from PerfDB based on the execution context, including question type, operator, prompt configuration, and model.
f_i^{fix}, f_i^{var}	Fixed (per-invocation) cost and variable (per-token) cost of invoking L_i.
ξ	Execution context (conditioning key for PERFDB); captures the conditions (e.g., topic, operator type, model) under which QoA/cost/latency/energy estimates are valid.
$\pi = (\{op_1, \ldots, op_m\}, \prec)$	Question plan; its operations (op_i is an LLM call or composition) and their partial order defining dependencies $\prec$.
$T_{op_i}^{out}$, $T_{op_i}^{proc}$	Number of tokens generated, and processed by an operation.
Seq($Q, L_1, \ldots, L_k$)	Sequential operation: L_2 takes L_1's answer to Q, Q itself, and a context prompt pt_{ctx} that instructs it on what to do.
Par($Q, \{L_1, \ldots, L_k\}$)	Parallel operation: each model process Q independently.
Blend(Q, Par($\ldots$), L_b)	Blending operation: model L_b combines outputs from a parallel execution, guided by blending prompt pt_{bld}.
$Financial_\pi$, $Latency_\pi$, $Energy_\pi$, QoA_π	Expected financial cost, latency, energy, and QoA of plan π.

total financial cost (USD/question), maximum total latency (seconds/question), maximum energy usage (J/question), and minimum acceptable QoA[2] (on a scale 0-1, 1 being the highest). We collectively refer to these as the *budget* $\mathcal{B}$. The weight vector $W = (w_F, w_L, w_E, w_{\text{QoA}})$, with $\sum_i w_i = 1$, encodes relative importance across objectives. Importantly, $\mathcal{B}$ defines *feasibility* (candidate plans must satisfy the user-constraints), whereas W is applied only after planning to select one plan from the Pareto-optimal set; W is not used to scalarize objectives during plan search. In practice, users do not need to specify all constraints manually. OPTI-Q can expose presets such as Cost-saving, Balanced, High-quality, Low-latency, and Energy-aware, which translate internally into default bounds and objective weights. We distinguish between two question types: (1) those with *binary correctness* (e.g., multiple-choice or yes/no questions), where $QoA \in \{0, 1\}$; and (2) those with *open-ended or free-text* answers, where $QoA \in [0, 1]$ reflects graded partial correctness (e.g., "John Smith III" vs. "J. Smith"). OPTI-Q's implementation provides estimators for QoA in both cases.

LLM Model. Let $L = \{L_1, L_2, \ldots, L_n\}$ be the available LLMs. Each L_i is represented by the tuple $(f_i, l_i, e_i, qoa_i, Tok_i, T_i^{\text{out}})$, where f_i, l_i, and e_i denote the expected financial cost, latency, and energy consumption of invoking L_i under an execution context; $Tok_i(\cdot)$ is the model-specific tokenizer that maps a prompt p_t to its input-token count $Tok_i(p_t)$ (e.g., GPT-4 vs. Llama-3 use different tokenizers); and T_i^{out} is the expected number of output tokens generated by L_i under the same context. We use ξ to denote the *execution context* (e.g., topic, operator type, and model). At planning time, Opti-Q instantiates the context-conditioned metrics $qoa_i(\xi)$, $f_i(\xi)$, $l_i(\xi)$, $e_i(\xi)$, and $T_i^{\text{out}}(\xi)$ [3] using the corresponding empirical estimates retrieved from PERFDB, rather than assuming a single global quality value per model. These metrics characterize the expected cost and behavior of model invocations within a plan. Each invocation cost includes a fixed and a variable component dependent on the number of input and output tokens under ξ (e.g., $f_i(\xi, p_t) = f_i^{\text{fix}}(\xi) + f_i^{\text{var}}(\xi) \cdot \big(Tok_i(p_t) + T_i^{\text{out}}(\xi)\big)$. Here, $f_i^{\text{fix}}(\xi)$ captures the minimum per-invocation cost, regardless of the specific question. To handle stochastic intermediate output lengths, PERFDB stores the expected output length $T_i^{\text{out}}(\xi)$ estimated from historical traces under the same context ξ, which Opti-Q uses to estimate token-dependent plan costs during planning.

Plan Model. For a question Q, let Π denote the set of admissible plans. A plan $\pi \in \Pi$ specifies how one or more LLMs from L are invoked to process Q. Each plan comprises operations $\{\text{op}_1, \ldots, \text{op}_n\}$, where each op_i represents either an LLM invocation or a composition of LLMs, and a partial order $\prec$ defining execution dependencies. If $\text{op}_i \prec \text{op}_j$, then op_j consumes the output of op_i. Initial operations, those with no predecessors, consume Q; final operations produce the plan's answer. Operations without order constraints will execute in parallel, and all operation outputs must either feed into subsequent operations or produce final answers. For example, a plan involving operations $\text{op}_1, \text{op}_2, \text{op}_3$ may specify $\text{op}_1 \prec \text{op}_2$, where both op_1 and op_3 execute in parallel and op_2 consumes op_1's output. A *sequential operation*, denoted $\text{Seq}(Q, L_1, \ldots, L_k) = L_k\left(Q \oplus p_{ctx} \oplus L_{k-1}\left(Q \oplus p_{ctx} \oplus \cdots \oplus L_1(Q)\right)\right)$, concatenates ($\oplus$) the question, a context prompt pt_{ctx} (that instructs the LLM on how to use it as a context; see Section V), and L_1's output for input to L_2. A *parallel operation*, denoted $\text{Par}(Q, \{L_1, \ldots, L_k\}) = L_1(Q) \odot L_2(Q) \odot \cdots \odot L_k(Q)$, poses Q to each model independently and uses structured output fusion ($\odot$) to combine their outputs. $\odot$ signifies a "concatenation" in which the output of each model is prefixed with its identifier for traceability (e.g., "LLM1: answer1; LLM2: answer2").

[2]We use the generic term *QoA* to denote any metric to assess model performance depending on the task and dataset, such as accuracy, BLEU, F1, or embedding-based similarity measures (e.g., cosine similarity).

[3]We may omit explicitly mentioning ξ when it is clear from context.

A *blending operation* combines multiple model outputs into a single answer [21]. This operation, denoted $\text{Blend}(Q, \text{Par}(Q, \dots), L_b) = L_b(Q \oplus pt_{bld} \oplus \text{Par}(Q, \dots))$, uses a dedicated "blending" model L_b, which takes the original question, a blending prompt p_{bld}, and the concatenated outputs of the parallel operation as input (Figure 1 shows an example; more details on the prompt in Section V). Blending operations may appear *only* after parallel operations, and each parallel operation must be immediately followed by exactly one blending operation. Note that the same model may appear multiple times in both sequential and parallel operations, so $\text{Seq}(Q, L_1, L_1)$ and $\text{Par}(Q, \{L_2, L_2\})$ are valid.

Multi-LLM QA Planning Problem. Let $\pi \in \Pi$ be a plan using a subset of LLMs of L to process Q. This plan induces total financial, latency, and energy consumption costs and obtains a QoA based on the chosen models and their composition. Our goal is to find Pareto-optimal plans balancing QoA, financial cost, latency, and energy under user constraints. Formally, we frame this as a MOO problem:

$$\text{Maximize} \quad [\text{QoA}(\pi), -\text{Financial}(\pi), -\text{Latency}(\pi), -\text{Energy}(\pi)]$$

$$\text{Subject to:} \quad \begin{cases} \text{Financial}(\pi) \le F_{\max} \\ \text{Latency}(\pi) \le L_{\max} \\ \text{Energy}(\pi) \le E_{\max} \\ \text{QoA}(\pi) \ge \text{QoA}_{\min} \end{cases}$$

B. Overview of OPTI-Q Approach

For a given question Q, our approach selects the plan $\pi \in \Pi$ that best balances QoA against financial, latency, and energy costs. As shown in Figure 1, consider the example question, *"Select all that apply: Which of the following are part of the Kingdom of Denmark? A) Faroe Islands B) Greenland C) Svalbard D) Lofoten Islands E) Aland Islands "*. First, OPTI-Q performs a **question parsing** phase to extract the prompt text pt, and topic(s) T (detailed in Section V), along with user-defined constraints and priorities (budget $\mathcal{B}$ and weight vector W). We require the user to specify $\mathcal{B}$; maximizing QoA "at any cost" is supported but not our focus. If $\mathbf{W}$ is omitted, we default to uniform weights.

Bounding the plan size. Next, OPTI-Q determines the maximum number of operations allowed per plan, k, based on $\mathcal{B}$. For each L_i, OPTI-Q computes the maximum model invocations n_i allowed under each constraint (dividing the relevant budget by model's per-constraint cost) and taking the minimum across constraints. For example, the budget in Figure 1 constrains a resource-heavy LLM, L_1, to one call while allowing a lighter L_2 up to three calls. Limits are not mutually exclusive: a plan may call L_1 once and still use L_2 one or more times within the remaining budget. OPTI-Q samples k uniformly from $[min_i(n_i), max_i(n_i)]$; as shown in our evaluation and in traditional ML ensemble size selection methods [36], this randomized choice within the interval does not harm performance.

Performance Statistics (PERFDB). OPTI-Q uses PERFDB, a statistical catalog that stores quality and resource statistics for individual LLM invocations and composed plans. PERFDB enables the planner to estimate plan metrics (i.e., latency, financial cost, QoA, and energy) without executing the plan. PERFDB is organized into levels of compositional depth (i.e., operator-nesting depth in the plan representation), which determines the granularity of stored statistics. Level 0 contains statistics for single LLMs obtained from public benchmarks (including ML.Energy [37] and Artificial Analysis[4]. Level 1 aggregates results of single operations with "depth 1", e.g., $\text{Seq}(Q, L_1, L_2)$ or $\text{Par}(Q, \{L_1, L_2, L_3\})$. Level 2 and higher, store measurements for subsets of plans with "depth 2" or higher (e.g., $\text{Blend}(Q, \text{Seq}(Q, L_1, L_2), \text{Par}(Q, \{L_3, L_4\}), L_b)$). All entries are keyed by execution context ξ, including topic, operator type, model, prompt configuration, and serving backend, thus capturing both workload-dependent behavior and backend-specific latency, energy, and throughput profiles. PERFDB is populated offline and incrementally after execution. For each executed plan, OPTI-Q appends a compact trace containing the execution context, plan structure, invocation configuration, token usage, and *observed* resource metrics. Cost and latency are obtained from API responses and execution logs. Energy is recorded when exposed by the backend, or *estimated* from token counts and prior measurements otherwise. QoA is updated only when quality feedback is available, such as benchmark labels, known reference answers, analyst judgments, user feedback, or automatic quality estimators. Thus, PERFDB stores *observed* values when available and *estimated* values otherwise. Since each execution contributes one compact trace, storage grows linearly with the number of logged executions, and PERFDB does not need to be rebuilt from scratch. New models, fine-tuned checkpoints, prompt variants, and serving backends are represented as distinct configured LLM

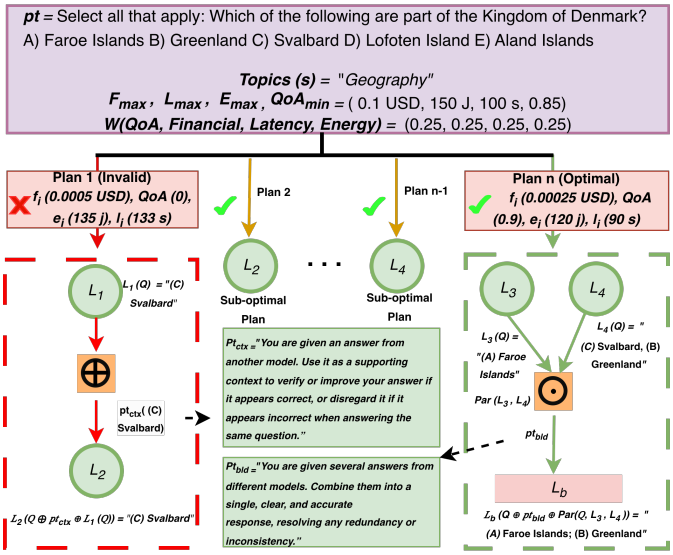


Fig. 1: Comparison of candidate multi-LLM execution plans for a question.

[4]https://artificialanalysis.ai/

invocation instances with initially sparse statistics; OPTI-Q bootstraps these instances from metadata or similar invocations and refines their estimates as new benchmark runs and live traces accumulate. Because LLM outputs and serving behavior may vary across runs, PERFDB also stores variance estimates and confidence intervals, enabling risk-aware planning when reliability is important. Section VI analyzes how varying PERFDB coverage affects planning performance.

Plan Generation. In this phase, OPTI-Q generates plans with up to k operations. Because the plan space grows exponentially in $|L|$ and k (as we will detail in the following section), an exhaustive search is infeasible. OPTI-Q therefore supports a "pluggable optimization engine" that includes diverse planning strategies (Section IV): (i) Dynamic Programming (DP), an exact solver that can recover the Pareto frontier for small $|L|$ and k but suffers from state-space explosion; (ii) stochastic Hill Climbing (HC), a lightweight greedy heuristic with low planning latency that can stagnate in local optima in our non-convex multi-objective landscape; and (iii) NSGA-II [38], a multi-objective evolutionary algorithm that balances exploration and exploitation. This design supports exact optimization when feasible, lightweight heuristics otherwise, and NSGA-II as the default scalable Pareto-front approximator.

Plan Execution Phase. To execute the selected plan, operations are carried out according to the plan topology, applying context propagation and blending where applicable. For example, consider the execution of *Plan* n in Figure 1, which includes a parallel call followed by blending, i.e., $\text{Blend}(Q, \text{Par}(Q, L_3, L_4), L_b)$, and is selected in this case. Models L_3 and L_4 run in parallel to answer the multi-answer user prompt. Suppose L_3 returns a partial but correct subset (e.g., (A) Faroe Islands), while L_4 returns (B) Greenland but also includes a plausible yet incorrect extra option (e.g., (C) Svalbard). The blender L_b aggregates the candidate sets, cross-checks them against the question constraints, removes unsupported options, and produces a consolidated final prediction (A, B) (Faroe Islands; Greenland). This simple example illustrates the strength of multi-objective planning: a single-model plan (only L_4) may be cheaper but less robust, risking error propagation. We quantify variability due to nondeterminism by repeated executions and confidence intervals in Section VI.

IV. PLAN GENERATION

To answer a question using multiple LLMs, we must (i) represent candidate execution plans in a compact form that supports efficient manipulation, (ii) estimate a plan's cost–benefit *without* executing it, and (iii) search the resulting plan space to find high-quality tradeoffs.

A. Plan Encoding

All planners in Opti-Q operate over the same compact, yet expressive encoding candidate plan π. We represent π as a tuple $(\textit{connectivity_map}, \mathbf{m})$, where *connectivity_map* canonically encodes the plan's directed acyclic execution topology and $\mathbf{m}$ assigns one model to each of the k nodes. Figure 2 shows an example plan with its encoding. The DAG topology over k operations is encoded by the upper-triangular adjacency matrix $M \in \{0,1\}^{k\times k}$, with $M_{ij} = 1$ for $(1 \le i < j \le k)$ iff there is a data-flow edge from L_i to L_j. For computational efficiency, we flatten M into a $k(k-1)/2$-bit vector $B = (b_{12}, b_{13}, \ldots, b_{(k-1)k})$, where each bit $b_{ij} \in \{0,1\}$ indicates whether L_i feeds its output to L_j. Blending operations are implicit: a designated blending model (L_b) is invoked to combine inputs if a node L_j has an in-degree $\deg^-(L_j) > 1$. To ensure invariance under node relabeling (structural isomorphism), we enumerate all valid topological orderings of the DAG and select the lexicographically maximal bit vector, B_{max}, defining the canonical integer identifier as *connectivity_map* $= \text{integer}(B_{\text{max}})$. This encoding method guarantees acyclicity by construction, facilitates efficient manipulation, and provides a unique, isomorphism-aware representation of complex execution flows. We enforce the constraint that the final node must have an in-degree ≥ 1, ensuring the plan produces a valid output.

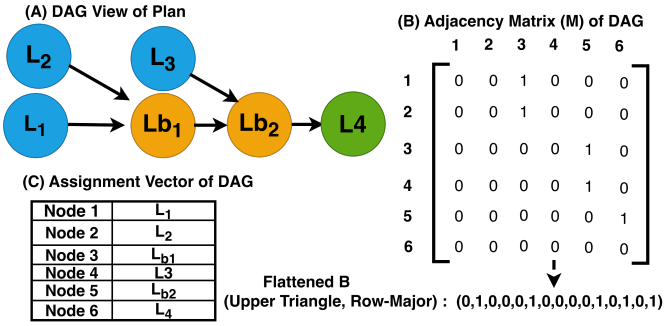


Fig. 2: Sample question plan encoding.

Figure 2(A) illustrates an execution plan composed of $k = 5$ operations constructed from Sequential, Parallel, and Blending operations, while Figure 2(B) and (C) show its connectivity matrix and model assignments, respectively. The plan begins with a parallel operation where models L_1 and L_2 process the question Q, defined as $op_1 = \text{Par}(Q, (L_1, L_2))$. Their outputs are then blended using model L_{b_1}, forming $op_2 = \text{Blend}(Q, op_1, L_{b_1})$. Concurrently, an independent parallel operation executes model L_3, defined as $op_3 = \text{Par}(Q, (L_3))$. The results from the first blend (op_2) and the execution of L_3 (op_3) are synthesized in a hierarchical blending step, $op_4 = \text{Blend}(Q, op_2, op_3, L_{b_2})$. Finally, in a sequential operation, $op_5 = \text{Seq}(Q, op_4, L_4)$, model L_4 processes the original question Q again using the output of op_4 as context to produce the final answer. This encoding captures the execution structure compactly and canonically, requiring $\mathcal{O}(k^2)$ space.

B. Cost-Benefit Estimation

For each candidate plan, we estimate (pre-execution) its expected QoA, financial cost, energy, and latency(see Algorithm 2).

Token Estimation. Cost-benefit metrics depend on the number of tokens processed and generated, which we must

estimate pre-execution. For each model L, let $Tok_L(\cdot)$ denote its tokenizer (models may segment the same string differently) and let T_L^{out} be the model's expected output length (e.g., average response tokens from PerfDB). We estimate two quantities for a plan π: total generated tokens T_π^{out} and total processed tokens T_π^{proc} (input + output across model invocations).

Sequential. In $\text{Seq}(Q, L_1, L_2)$, L_1 consumes $Tok_{L_1}(Q)$ tokens and produces $T_{L_1}^{\text{out}}$. The second model consumes the original question plus a context prompt pt_{ctx} and the *immediate predecessor's* output, i.e., $Tok_{L_2}\big(Q + pt_{\text{ctx}} + T_{L_1}^{\text{out}}\big)$, and produces $T_{L_2}^{\text{out}}$. We conservatively sum outputs, $T_{\text{Seq}}^{\text{out}} = T_{L_1}^{\text{out}} + T_{L_2}^{\text{out}}$, allowing for potential redundancy where later models restate prior text. For a length-k chain $\text{Seq}(Q, L_1, \ldots, L_k)$ we avoid exponential growth by appending only the immediate predecessor's output at each step: $T_\pi^{\text{proc}} = \big(Tok_{L_1}(Q) + T_{L_1}^{\text{out}}\big) + \sum_{i=2}^{k}\Big(Tok_{L_i}\big(Q + pt_{\text{ctx}} + T_{L_{i-1}}^{\text{out}}\big) + T_{L_i}^{\text{out}}\Big)$. Equivalently, chains can be viewed as nested compositions, e.g., $\text{Seq}(Q, L_1, L_2, L_3) = \text{Seq}(Q, \text{Seq}(Q, L_1, L_2), L_3)$, with pt_{ctx} applied from the second step onward.

Parallel + blending. In $\text{Blend}\big(Q, \text{Par}(Q, (L_1, \ldots, L_k)), L_b\big)$, each L_i processes $Tok_{L_i}(Q)$ input tokens and produces $T_{L_i}^{\text{out}}$ output tokens. The blending model L_b consumes the question, a blending prompt pt_{bld}, and the concatenated outputs: $Tok_{L_b}\Big(Q + pt_{\text{bld}} + \sum_{i=1}^{k} T_{L_i}^{\text{out}}\Big)$ and produces $T_{L_b}^{\text{out}}$. Thus, $T_\pi^{\text{proc}} = \sum_{i=1}^{k}\big(Tok_{L_i}(Q) + T_{L_i}^{\text{out}}\big) + Tok_{L_b}\Big(Q + pt_{\text{bld}} + \sum_{i=1}^{k} T_{L_i}^{\text{out}}\Big) + T_{L_b}^{\text{out}}, \quad T_\pi^{\text{out}} = T_{L_b}^{\text{out}}$.

General plans. For an arbitrary (nested) plan π, we compute $(T_\pi^{\text{proc}}, T_\pi^{\text{out}})$ by recursively applying the above rules bottom-up over the plan tree/DAG and summing the processed tokens over all invoked models. This yields a conservative, tokenizer-aware estimate of token volume that propagates through both sequential and parallel compositions.

For the nested plan (Figure 2),

$$\text{Seq}\big(Q,\ \text{Blend}\big(Q,\ \text{Par}\big(Q,\ (L_3,\ \text{Blend}\big(Q,\ \text{Par}(Q, (L_1, L_2)),\ L_{b_1}\big)\big),\ L_{b_2}\big),\ L_4\big)\big)$$

tokens are accumulated from the innermost operation outward. For clarity, we assume that all models share the same tokenizer. We begin with the parallel operation $\text{Par}(Q, (L_1, L_2))$, where both L_1 and L_2 process the question Q ($Tok_{L_1}(Q) = Tok_{L_2}(Q) = 100$) and produce $T_{L_1}^{\text{out}} = 50$ and $T_{L_2}^{\text{out}} = 60$ tokens, respectively. Their combined outputs (110 tokens) are passed to the blending model L_{b_1}, which also receives the original question and a blending prompt ($pt_{\text{bld}} = 20$), yielding a total input of $Tok_{L_{b_1}}(Q + pt_{\text{bld}} + T_{L_1}^{\text{out}} + T_{L_2}^{\text{out}}) = 230$ tokens and producing $T_{L_{b_1}}^{\text{out}} = 100$ tokens. This blended output then feeds into another blending operation L_{b_2} together with model L_3, which processes Q independently ($Tok_{L_3}(Q) = 100$ and $T_{L_3}^{\text{out}} = 150$, giving $T_{L_3}^{\text{proc}} = 250$). The two outputs—$L_{b_1}$'s 100 tokens and L_3's 150 tokens—are then fused by L_{b_2}, which processes $Tok_{L_{b_2}}(Q + pt_{\text{bld}} + T_{L_{b_1}}^{\text{out}} + T_{L_3}^{\text{out}}) = 490$ tokens and produces $T_{L_{b_2}}^{\text{out}} = 120$ tokens. Finally, in the outer sequential operation, model L_4 processes the question Q, the output from L_{b_2}, and a context prompt ($pt_{\text{ctx}} = 30$), for a total of $Tok_{L_4}(Q + pt_{\text{ctx}} + T_{L_{b_2}}^{\text{out}}) = 400$ tokens, and produces $T_{L_4}^{\text{out}} = 150$ tokens. Summing all components, the plan processes $\approx 1{,}780$ tokens, illustrating how token counts propagate through composition.

QoA Estimation. Predicting a plan's QoA before execution is difficult due to model nondeterminism and the combinatorial space of multi-LLM plans. We therefore estimate QoA using PERFDB, a performance database populated from offline benchmarks and execution traces that stores topic-conditioned QoA statistics for (i) atomic invocations (e.g., (Q, L_i)) and (ii) common local compositions (e.g., $Seq(Q, L_i, L_j)$ and $Blend(Q, Par(Q, \{L_1, \ldots, L_k\}), L_b)$). The goal is not perfect calibration, but a conservative estimate that preserves *relative* differences between candidate plans, which is sufficient for Pareto-based search.

Topic conditioning and lookup. Let $\mathcal{T} = \{T_1, \ldots, T_m\}$ be the inferred topics of Q. For any operator or substructure *op*, we aggregate topic-specific entries as $QoA(op) = \frac{1}{m}\sum_{i=1}^{m} QoA_{op,T_i}$. To initialize estimation, we first attempt an exact PERFDB match for the whole plan (or large substructures). If not available, we perform a fuzzy match using a weighted similarity that combines structural overlap with a normalized edit-distance over a canonicalized plan string; we accept a fuzzy match only if the score exceeds a fixed threshold (e.g., 0.75 in our experiments). Importantly, all QoA values used during planning are retrieved from PERFDB under a single schema (topic + operator/substructure key), even when the retrieval itself is fuzzy. The complete process is summarized in Algorithm 1.

Sequential refinement. For a sequential dependency where op_j consumes the output of op_i (e.g., a local $\text{Seq}(\cdot, L_i, L_j)$), we update the running estimate using a pairwise relative-effect factor derived from PERFDB: $QoA_{\text{next}} = \text{clip}_{[0,1]}\Big(QoA_{\text{curr}} \cdot \frac{QoA(Q, \text{Seq}(L_i, L_j))}{\max(\epsilon, QoA(Q, L_j))}\Big)$. where ϵ is a small constant and $\text{clip}_{[0,1]}(\cdot)$ bounds the result to $[0, 1]$. Intuitively, the ratio compares the composed step against the successor alone, capturing the marginal impact of inserting L_i before L_j under the same topic-conditioned context. We adopt a multiplicative relative-effect formulation because composition effects in our traces scale with the successor's baseline quality (i.e., prepending a context-providing model tends to improve a weak successor proportionally more than a strong one) which an additive model does not capture. The $\text{clip}_{[0,1]}(\cdot)$ guards the small fraction of cases where sparse-coverage ratios extrapolate beyond the valid range. *Example:* if $QoA(Q, L_j) = 0.50$ and $QoA(Q, \text{Seq}(L_i, L_j)) = 0.60$, the factor is 1.2; with $QoA_{\text{curr}} = 0.55$, we obtain $QoA_{\text{next}} \approx \text{clip}_{[0,1]}(0.55 \times 1.2) = 0.66$.

Parallel + blending. For a blend node op_{L_b}, where the blending operation is defined as $\text{Blend}(Q, \text{Par}(Q, \{L_1, \ldots, L_k\}), L_b)$, we combine the incoming branch estimates using PERFDB references for (i) the blend operator itself and (ii) each predecessor run standalone. Let $QoA^{\text{ref}}_{op_{L_b}} = QoA(Q, op_{L_b})$ and $QoA^{\text{ref}}_{L_s} = QoA(Q, L_s)$. Given the branch estimates computed in the current traversal, $QoA^{\text{new}}_{L_s}$, we scale the reference blend quality by the average relative change of its

inputs:

$$QoA^{\text{new}}_{op_{L_b}} = \text{clip}_{[0,1]}\left(QoA^{\text{ref}}_{op_{L_b}} \cdot \left(1 + \frac{1}{k}\sum_{s=1}^{k} \frac{QoA^{\text{new}}_{L_s} - QoA^{\text{ref}}_{L_s}}{\max(\epsilon,\, QoA^{\text{ref}}_{L_s})}\right)\right).$$

This estimator is robust to a single weak branch (average rather than product) while still reflecting systematic improvements/degradations across branches. Note that the averaging form estimates the expected blend quality and is deliberately agnostic to which branch is correct; it does not model the error-correction mechanism illustrated in Figure 1, which is realized at execution time. This is a conservative choice that avoids over-crediting blends on the basis of a single strong branch at planning time. *Example:* suppose $k = 2$, $QoA^{\text{ref}}_{op_{L_b}} = 0.60$, and references $QoA^{\text{ref}}_{L_1} = 0.50$, $QoA^{\text{ref}}_{L_2} = 0.40$. If the current plan yields $QoA^{\text{new}}_{L_1} = 0.60$ and $QoA^{\text{new}}_{L_2} = 0.44$, then the average relative change is $\frac{1}{2}\left(\frac{0.60-0.50}{0.50} + \frac{0.44-0.40}{0.40}\right) = 0.15$, so $QoA^{\text{new}}_{op_{L_b}} = \text{clip}(0.60 \times (1 + 0.15)) = 0.69$.

Plan-level propagation and fallbacks. We compute the plan QoA by traversing the plan DAG in a topological order that respects precedence constraints (see Algorithm 1). So, an operator is processed only after all prerequisites are estimated. Along edges corresponding to sequential consumption we apply the sequential update rule; at blend nodes we wait for all incoming branches and apply the blending rule. When PERFDB lacks a required composed entry (e.g., missing $QoA(Q, \text{Seq}(L_i, L_j))$), we fall back to the closest available statistics (e.g., use other known predecessors of L_j to estimate an average relative-effect factor, or default to a neutral factor of 1 when no pairwise information exists). For nested blend structures where direct references are sparse, we normalize the structure by flattening nested blends before performing lookups, then apply the same blend rule on the resulting fan-in. These fallbacks ensure estimation completes under incomplete coverage; in Section VI we quantify how estimation error varies with PERFDB coverage.

Algorithm 1 summarizes the preprocessing stage used before traversal-based QoA estimation. Its purpose is to populate two maps from PERFDB: *Matched*, which stores metrics for whole-plan or subplan matches keyed by sink node, and *BlendRef*, which stores the reference metrics required by the blend estimator. The procedure first attempts an exact or fuzzy lookup of the entire plan. If no full-plan match is found, it searches for maximal sequential sub-chains and stores any successful matches in *Matched*. It then constructs *BlendRef* by collecting, for each blend node, both the reference metrics of the blend operator itself and the standalone reference metrics of its predecessor inputs. These outputs are later consumed by the traversal procedure in Appendix A.

Financial Cost Estimation. We estimate the financial cost of a plan as the sum of its operation-level costs [29]. For a single invocation of model L_i, we use a simple fixed-plus-variable pricing model: a fixed per-call charge f_i^{fix} and a per-token rate f_i^{var} applied to the total number of billed tokens (prompt + generated). Using our token estimator, the expected billed tokens for op_i equal the model-specific tokenized prompt length plus the predicted output length: $Financial(Q, L_i) = f_i^{\text{fix}} + f_i^{\text{var}} \cdot \left(Tok_i(p_t) + T^{\text{out}}_{op_i}\right)$. For a parallel operator $\text{Par}(Q, \{L_1, \ldots, L_n\})$, invocations are independent, so costs add: $Financial(\text{Par}(Q, \{L_1, \ldots, L_n\})) = \sum_{i=1}^{n} Financial(Q, L_i)$. For a general plan π, we account for prompt growth in sequential and nested structures by charging each operation for its own prompt plus the outputs of its immediate predecessors: $Financial(\pi) = \sum_{i=1}^{n}\left(f_i^{\text{var}}\left(Tok_i(p_t) + \sum_{op_j \in \text{pred}(op_i)} T^{\text{out}}_{op_j} + T^{\text{out}}_{op_i}\right) + f_i^{\text{fix}}\right)$. Here, $\text{pred}(op_i)$ denotes the set of operators whose outputs are included in op_i's prompt (e.g., the immediate predecessor in a sequence, or all fan-in branches for a blend node).

Energy Consumption Estimation. LLM energy consumption is approximately proportional to the number of tokens processed and generated [39]. Let e_i (J/token) denote the

Algorithm 1 PROCESS PLAN

Require: DAG $G = (V, E)$, question Q (for topic T), fuzzy threshold s, historical performance database PERFDB
Ensure: Two maps: $Matched$ (sink node $\mapsto$ metrics object) and $BlendRef$ (blend node $\mapsto$ map[input node $\mapsto$ metrics object])

Phase 0 – validation
if not ISVALIDDAG(G) **then**
  raise INVALIDDAGERROR
end if
$Matched \leftarrow \varnothing$; $BlendRef \leftarrow \varnothing$
Phase 1 – whole-graph lookup
$\mathcal{X} \leftarrow$ PERFDBLOOKUPEXACT(G, Q) ▷ Lookup based on graph structure and question topics
if $\mathcal{X} = \varnothing$ **then**
  $\mathcal{X} \leftarrow$ PERFDBLOOKUPFUZZY(G, Q, s) ▷ Fuzzy match based on structure, semantics, threshold s
end if
if $\mathcal{X} \neq \varnothing$ **then**
  $Matched$[SINK(G)] $\leftarrow \mathcal{X}$ ▷ Store metrics object
  return $(Matched, BlendRef)$ ▷ Return early if full match found
end if
Phase 2 – sequential sub-chains
$S \leftarrow$ STARTNODES(G)
for all $C \in$ FINDMAXIMALSEQCHAINS(G, S) **do** ▷ C is a DAG representing one chain
  $\mathcal{X} \leftarrow$ PERFDBLOOKUPEXACT(C, Q)
  if $\mathcal{X} = \varnothing$ **then**
    $\mathcal{X} \leftarrow$ PERFDBLOOKUPFUZZY(C, Q, s) ▷ Recursive fuzzy match on sub-chains if needed
  end if
  if $\mathcal{X} \neq \varnothing$ **then**
    $Matched$[SINK(C)] $\leftarrow \mathcal{X}$ ▷ Store metrics for the matched chain
  end if
end for
Phase 3 – blend reference table
$G' \leftarrow$ FLATTENNESTEDBLENDS(G) ▷ Optional: Simplify structure if needed
for all $b \in$ BLENDNODES(G') **do**
  $\mathcal{X}_{\text{blend}} \leftarrow$ DBLOOKUPPARALLELREF(b, Q) ▷ Reference metrics for blend op b
  $BlendRef[b]$['ref_blend'] $\leftarrow \mathcal{X}_{\text{blend}}$
  for all $u \in$ PREDECESSORS(G', b) **do**
    $\mathcal{X}_{\text{in}} \leftarrow$ DBLOOKUPSINGLENODE(u, Q) ▷ Reference metrics for input u (standalone)
    if $\mathcal{X}_{\text{in}} \neq \varnothing$ **then**
      $BlendRef[b][u] \leftarrow \mathcal{X}_{\text{in}}$ ▷ Store standalone metrics for input u
    end if
  end for
end for
return $(Matched, BlendRef)$

energy per token for operation op_i obtained from PERFDB or external measurements [37]). We estimate plan energy by reusing the same structural accounting as financial cost, but without a fixed per-call term. This keeps energy estimation consistent with how prompts and intermediate outputs propagate through the plan.

Latency Estimation. Plan latency depends on both the plan structure and the number of tokens processed and generated [32]. For a single invocation op_i, we approximate latency as linear in token volume using a per-token time coefficient l_i (s/token) $Latency(Q, L_i) = l_i(Tok_i(p_t)+T^{\text{out}}_{op_i})$. For sequential (or more generally, precedence-constrained) portions of a plan, latencies add along executed operations, using the same prompt-growth accounting as above. For a pure parallel operator $\text{Par}(Q, \{L_1, \ldots, L_n\})$, branches execute concurrently, so the end-to-end latency is dominated by the slowest branch. For mixed plans with parallelism and joins, we apply these rules along the plan DAG: sum along each critical path and take the maximum over paths that run concurrently.

C. Pareto Search over Multi-LLM Plans

The plan space is inherently combinatorial. Under our operator-DAG representation, each node selects one of $|L|$ models and each of the $\binom{k}{2}$ forward edges may be present or absent, yielding $|\Pi| = |L|^k \cdot 2^{\binom{k}{2}} = |L|^k \cdot 2^{\frac{k(k-1)}{2}}$, with the derivation provided next. *(Model assignments)* For each of the k ordered stages, we choose any model in L, with reuse permitted. By independence across stages, the number of stage-to-model assignments is

$$N_{\text{assign}} \;=\; |L|^k.$$

(Acyclic topologies) With the node order fixed as $1 < \cdots < k$, a directed edge may only go from i to j when $i < j$. There are exactly

$$N_{\text{edges}} \;=\; \binom{k}{2} \;=\; \frac{k(k-1)}{2}$$

eligible edges, each chosen independently to be present or absent. Hence, the number of admissible upper-triangular (and therefore acyclic) edge sets is

$$N_{\text{conn}} \;=\; 2^{\binom{k}{2}} \;=\; 2^{\frac{k(k-1)}{2}}.$$

(Independence and product rule) Model choices and edge choices are independent design dimensions, so

$$|\Pi| \;=\; N_{\text{assign}} \cdot N_{\text{conn}} \;=\; |L|^k \cdot 2^{\frac{k(k-1)}{2}}.$$

a) Remarks.: (i) This is a labeled count with execution order $1<\ldots<k$ fixed; it therefore upper-bounds implementations that later canonicalize isomorphic topologies or impose additional syntactic constraints (e.g., requiring a single sink or immediate blending at fan-ins). (ii) The expression makes the combinatorial growth explicit:

$$\log|\Pi| = k\log|L| + \frac{k(k-1)}{2}\log 2,$$

we present the core mechanics, full plan-generation details and algorithmic variants below.

Genetic Algorithm (NSGA-II). NSGA-II is a robust default for our discrete, non-convex search space spanning both topology and model assignment. As summarized in Algorithm 2, each individual encodes (i) a canonicalized DAG topology (`connectivity_map`) and (ii) a model-assignment vector $\mathbf{m}$ specifying one LLM per node. Starting from an initial population of size N, we first evaluate all individuals using QUERYORPREDICT over PERFDB and mark infeasible plans based on the user-specified cost, latency, energy, and QoA constraints (lines 1–7). NSGA-II then iterates for G generations using binary tournament selection based on non-dominated rank and crowding distance (lines 8–11). With crossover probability C_r, we apply one-point crossover independently to the topology and assignment segments, followed by targeted mutations that mirror common plan edits: (i) edge flips (rewiring), (ii) node insertions or deletions (operator modification), and (iii) model reassignment on non-blending nodes (lines 12–18). After any structural edit, we apply lightweight repair to enforce validity, including a valid sink/output and the blending constraint that any node with in-degree greater than one must use the blending model. We then canonicalize the resulting plan so that isomorphic workflows map to a single encoding. Offspring are re-evaluated through QUERYORPREDICT and feasibility checking (lines 20–24), after which the parent and offspring populations are merged and ranked using fast non-dominated sorting with constraint handling, where feasible solutions dominate infeasible ones and infeasible solutions are ordered by total violation (lines 25–27). The first non-dominated front $\mathcal{F}_1$ provides an approximation to the Pareto set, from which we select a single execution plan by applying the user preference vector W to normalized objectives (line 28).

Dynamic Programming. The DP baseline uses the same canonical bit-plan representation, objective definitions, constraints, and final preference-based selection rule as NSGA-II. As summarized in Algorithm 3, DP starts from a set of seed plans and incrementally constructs larger plans up to depth k by applying valid structural and model-assignment edits. Each candidate is evaluated using QUERYORPREDICT over PERFDB, with the same objective tuple $(\text{Cost}, \text{Latency}, \text{Energy}, \text{QoA})$ and the same user-specified bounds $(F_{\max}, L_{\max}, E_{\max}, \text{QoA}_{\min})$ used throughout the framework.

To control state growth, we apply configurable post-expansion pruning at each depth (Algorithm 3, lines 18–23). Our implementation supports three modes: no pruning, parent-specific Δ-QoA progress pruning, and approximate dominance pruning via multiplicative ε-dominance following [40]. Table II summarizes the resulting runtime–quality trade-offs. Unless otherwise stated, we use the parent-specific Δ-QoA rule by default, retaining a child only if $QoA(\pi_{\text{child}}) \geq QoA(\pi_{\text{parent}}) + \Delta$, with $\Delta = 0.05$. This setting provided a strong runtime–quality trade-off in our analysis, substantially reducing runtime while preserving high hypervolume. After

Algorithm 2 NSGA-II for multi-objective schedule optimization.

Require: Initial population $\mathcal{P}_0$, population size $N = |\mathcal{P}_0|$, generations G, crossover probability C_r,
mutation operators $\mu \leftarrow \{\mu_1$: edge mod., μ_2 : node mod., μ_3 : LLM adj.$\}$,
constraints $(F_{\max}, L_{\max}, E_{\max}, \mathrm{QoA}_{\min})$, performance database PerfDB,
user preference vector $W = (w_C, w_L, w_E, w_{\mathrm{QoA}})$
Ensure: Schedule selected from the first Pareto front based on W
for all $\pi_i \in \mathcal{P}_0$ **do** ▷ Initial population evaluation
  $(\mathrm{QoA}_i, \mathrm{Cost}_i, \mathrm{Latency}_i, \mathrm{Energy}_i) \leftarrow$ QUERYORPREDICT(π_i, PERFDB)
  if $\mathrm{Cost}_i > F_{\max}$ **OR** $\mathrm{Latency}_i > L_{\max}$ **OR** $\mathrm{Energy}_i > E_{\max}$ **OR** $\mathrm{QoA}_i < \mathrm{QoA}_{\min}$ **then**
    Mark π_i as infeasible
  end if
  $\pi_i.\mathrm{obj} \leftarrow (-\mathrm{Cost}_i, -\mathrm{Latency}_i, -\mathrm{Energy}_i, \mathrm{QoA}_i)$ ▷ Store objectives for minimization
end for
for $g \leftarrow 0$ **TO** $G-1$ **do** ▷ Main evolutionary loop
  $\{\mathcal{F}_1, \mathcal{F}_2, \dots\} \leftarrow$ FASTNONDOMINATEDSORT($\mathcal{P}_g$) ▷ Rank based on dominance
  Calculate CROWDINGDISTANCE($\mathcal{F}_k$) for all fronts $\mathcal{F}_k$ ▷ Calculate diversity
  $\mathcal{M} \leftarrow$ BINARYTOURNAMENTSELECTION($\mathcal{P}_g, N$) ▷ Select mating pool
  $\mathcal{Q}_g \leftarrow \emptyset$ ▷ Initialize offspring population
  while $|\mathcal{Q}_g| < N$ **do** ▷ Generate offspring
    $(S_p, S_q) \leftarrow$ SELECTPARENTS($\mathcal{M}$)
    $(O_1, O_2) \leftarrow \begin{cases} \text{CROSSOVER}(S_p, S_q) & \textbf{if } \text{RAND}() < C_r \\ (S_p, S_q) & \textbf{otherwise} \end{cases}$
    $O_1 \leftarrow$ MUTATE(O_1, μ); ▷ Apply mutation strategies
    $O_2 \leftarrow$ MUTATE(O_2, μ);
    $\mathcal{Q}_g \leftarrow \mathcal{Q}_g \cup \{O_1, O_2\}$
  end while
  for all $\pi_i \in \mathcal{Q}_g$ **do** ▷ Evaluate offspring
    $(\mathrm{QoA}_i, \mathrm{Cost}_i, \mathrm{Latency}_i, \mathrm{Energy}_i) \leftarrow$ QUERYORPREDICT(π_i, PERFDB)
    if $\mathrm{Cost}_i > F_{\max}$ **OR** $\mathrm{Latency}_i > L_{\max}$ **OR** $\mathrm{Energy}_i > E_{\max}$ **OR** $\mathrm{QoA}_i < \mathrm{QoA}_{\min}$ **then**
      Mark π_i as infeasible ▷ Re-check feasibility
    end if
    $\pi_i.\mathrm{obj} \leftarrow (-\mathrm{Cost}_i, -\mathrm{Latency}_i, -\mathrm{Energy}_i, \mathrm{QoA}_i)$ ▷ Update objectives
  end for
  $\mathcal{R}_g \leftarrow \mathcal{P}_g \cup \mathcal{Q}_g$ ▷ Combine parent and offspring
  $\{\mathcal{F}_1, \mathcal{F}_2, \dots\} \leftarrow$ FASTNONDOMINATEDSORT($\mathcal{R}_g$) ▷ Rank combined population
  $\mathcal{P}_{g+1} \leftarrow$ SELECTNEXTGENERATION($\{\mathcal{F}_k\}, N$) ▷ Select based on rank and crowding
end for
return SELECTBYPREFERENCE(GETFRONT($\mathrm{P}_G, 1$), W) ▷ Select from best front using user weights

canonical deduplication and pruning, the final plan is selected from the first Pareto front using the user preference vector W.

Hill Climbing. Because DP can become state-intensive as k grows, we also implement a randomized local-search optimizer that quickly finds high-quality trade-offs under a per-question time budget, following [41]. As summarized in Algorithm 4, the method performs repeated random restarts: each restart samples a feasible plan π, then greedily hill-climbs through local plan edits until reaching a locally non-dominated solution. Collecting these local optima across restarts yields an approximate Pareto frontier.

The optimizer uses the same canonical bit-plan representation, objective definitions, constraints, and final preference-based selection rule as NSGA-II and DP. The neighborhood of a plan π is defined by exhaustive single-step mutations: (i) *model mutation*, which changes the base model at one non-blending node; (ii) *edge flip*, which toggles one edge in the upper-triangular encoding and retains the result only if it remains a valid DAG; and (iii) *node addition*, which appends one new node (up to the operation limit k) using an admissible extension that preserves a valid sink/output. After any structural mutation, we repair assignments to enforce the blending constraint, canonicalize the topology, and remap $\mathbf{m}$ under the canonical permutation so that isomorphic DAGs share a single encoding.

Each candidate is evaluated using QUERYORPREDICT over PERFDB and filtered using the same hard bounds $(F_{\max}, L_{\max}, E_{\max}, \mathrm{QoA}_{\min})$ as the other planners. A neigh-

Algorithm 3 Dynamic programming for multi-objective schedule optimization.

Require: Seed plans $\mathcal{S}_0$, maximum operations k, pruning mode $P \in \{\texttt{none}, \texttt{delta_qoa}, \texttt{epsilon}\}$,
optional pruning parameter Δ or ε, constraints $(F_{\max}, L_{\max}, E_{\max}, \mathrm{QoA}_{\min})$,
performance database PerfDB, preference vector $W = (w_C, w_L, w_E, w_{\mathrm{QoA}})$
Ensure: Schedule selected from the final Pareto set according to W
$\mathcal{B}_1 \leftarrow \mathcal{S}_0$ ▷ Initialize bucket of 1-operation plans
for $d \leftarrow 1$ **TO** k **do**
  for all $\pi_i \in \mathcal{B}_d$ **do**
    $(\mathrm{QoA}_i, \mathrm{Cost}_i, \mathrm{Latency}_i, \mathrm{Energy}_i) \leftarrow$ QUERYORPREDICT(π_i, PerfDB)
    if $\mathrm{Cost}_i > F_{\max}$ **OR** $\mathrm{Latency}_i > L_{\max}$ **OR** $\mathrm{Energy}_i > E_{\max}$ **OR** $\mathrm{QoA}_i < \mathrm{QoA}_{\min}$ **then**
      Mark π_i as infeasible
    end if
    $\pi_i.\mathrm{obj} \leftarrow (-\mathrm{Cost}_i, -\mathrm{Latency}_i, -\mathrm{Energy}_i, \mathrm{QoA}_i)$
  end for
  if $d = k$ **then**
    break
  end if
  $\mathcal{B}_{d+1} \leftarrow \emptyset$
  for all $\pi_p \in \mathcal{B}_d$ **do** ▷ Expand all parents at depth d
    $\mathcal{C} \leftarrow$ EXPAND(π_p)
    for all $\pi_c \in \mathcal{C}$ **do**
      $\pi_c \leftarrow$ REPAIRANDCANONICALIZE(π_c)
      $(\mathrm{QoA}_c, \mathrm{Cost}_c, \mathrm{Latency}_c, \mathrm{Energy}_c) \leftarrow$ QUERYORPREDICT(π_c, PerfDB)
      if $\mathrm{Cost}_c > F_{\max}$ **OR** $\mathrm{Latency}_c > L_{\max}$ **OR** $\mathrm{Energy}_c > E_{\max}$ **OR** $\mathrm{QoA}_c < \mathrm{QoA}_{\min}$ **then**
        continue
      end if
      $\pi_c.\mathrm{obj} \leftarrow (-\mathrm{Cost}_c, -\mathrm{Latency}_c, -\mathrm{Energy}_c, \mathrm{QoA}_c)$
      $\mathcal{B}_{d+1} \leftarrow \mathcal{B}_{d+1} \cup \{(\pi_p, \pi_c)\}$ ▷ Keep parent-child link for pruning
    end for
  end for
  $\mathcal{B}_{d+1} \leftarrow$ DEDUPLICATECANONICAL($\mathcal{B}_{d+1}$)
  if $P = \texttt{delta_qoa}$ **then**
    $\mathcal{B}_{d+1} \leftarrow$ DELTAQOAPRUNE($\mathcal{B}_{d+1}, \Delta$)
  else if $P = \texttt{epsilon}$ **then**
    $\mathcal{B}_{d+1} \leftarrow$ EPSILONPRUNE($\mathcal{B}_{d+1}, \varepsilon$)
  end if
  $\mathcal{B}_{d+1} \leftarrow$ PARETOPRUNE($\mathcal{B}_{d+1}$)
end for
$\mathcal{R} \leftarrow \bigcup_{d=1}^{k} \mathcal{B}_d$
$\mathcal{F}_1 \leftarrow$ GETFRONT($\mathcal{R}, 1$)
return SELECTBYPREFERENCE($\mathcal{F}_1, W$)

TABLE II: Runtime–quality trade-off of DP pruning strategies at $k = 5$, averaged over 10 topics. Hypervolume (HV) is reported as a percentage of the no-pruning baseline.

$\varepsilon = \Delta$	No pruning		ε-dominance		Δ-QoA gate	
	HV (%)	Time (s)	HV (%)	Time (s)	HV (%)	Time (s)
0.01	100.0	227.5	95.5	22.8	96.3	14.5
0.02	100.0	227.5	95.3	20.2	95.8	9.5
0.05	100.0	227.5	93.3	11.6	92.3	2.1
0.10	100.0	227.5	90.3	7.7	78.8	0.2
0.20	100.0	227.5	83.0	3.7	68.9	0.0

bor is accepted if it dominates the current plan, i.e., it is no worse on all objectives and strictly better on at least one. We scan the neighborhood and take the first dominating move, repeating until no neighbor dominates the current plan (Algorithm 4, lines 9–19). As locally optimal candidates are discovered, we maintain a global frontier across restarts; optionally, this insertion step can apply the same parent-specific Δ-QoA filter used in DP.

Algorithm 4 Hill climbing for multi-objective schedule optimization.

Require: Time budget T, maximum operations k, constraints $(F_{\max}, L_{\max}, E_{\max}, \mathrm{QoA}_{\min})$,
1: performance database PerfDB, preference vector $W = (w_C, w_L, w_E, w_{\mathrm{QoA}})$
Ensure: Schedule selected from the final approximate Pareto frontier according to W
2: $\mathcal{F} \leftarrow \emptyset$ ▷ Global frontier
3: **while** TIMEREMAINING(T) **do**
4: $\pi \leftarrow$ SAMPLEFEASIBLEPLAN(k) ▷ Random restart
5: $(\mathrm{QoA}, \mathrm{Cost}, \mathrm{Latency}, \mathrm{Energy}) \leftarrow$ QUERYORPREDICT(π, PerfDB)
6: $\pi.\mathrm{obj} \leftarrow (-\mathrm{Cost}, -\mathrm{Latency}, -\mathrm{Energy}, \mathrm{QoA})$
7: improved $\leftarrow$ **true**
8: **while** improved **do**
9: improved $\leftarrow$ **false**
10: $\mathcal{N} \leftarrow$ GENERATENEIGHBORHOOD(π, k)
11: **for all** $\pi' \in \mathcal{N}$ **do**
12: $\pi' \leftarrow$ REPAIRANDCANONICALIZE(π')
13: $(\mathrm{QoA}', \mathrm{Cost}', \mathrm{Latency}', \mathrm{Energy}') \leftarrow$ QUERYORPREDICT(π', PerfDB)
14: **if** $\mathrm{Cost}' > F_{\max}$ **OR** $\mathrm{Latency}' > L_{\max}$ **OR** $\mathrm{Energy}' > E_{\max}$ **OR** $\mathrm{QoA}' < \mathrm{QoA}_{\min}$ **then**
15: **continue**
16: **end if**
17: $\pi'.\mathrm{obj} \leftarrow (-\mathrm{Cost}', -\mathrm{Latency}', -\mathrm{Energy}', \mathrm{QoA}')$
18: **if** DOMINATES(π', π) **then**
19: $\pi \leftarrow \pi'$ ▷ First improving move
20: improved $\leftarrow$ **true**
21: **break**
22: **end if**
23: **end for**
24: **end while**
25: $\mathcal{F} \leftarrow$ INSERTINTOFRONTIER($\mathcal{F}, \pi$)
26: **end while**
27: $\mathcal{F}_1 \leftarrow$ GETFRONT($\mathcal{F}, 1$)
28: **return** SELECTBYPREFERENCE($\mathcal{F}_1, W$)

V. OPTI-Q IMPLEMENTATION

We implement OPTI-Q as a modular framework for flexible multi-LLM QA planning under user-defined constraints[5]. Our prototype integrates five widely used open-source LLMs: Gemma-3:27B [42], LLaMA3-ChatQA (8B) [43], Qwen2.5 (14B) [44], Phi-4 (14B) [45], and Mistral (7B) [46]. We selected these models for their complementary strengths across benchmarks [15], [47] (e.g., strong reasoning vs. conversational QA vs. efficiency). All models run locally via Ollama[6], which also simplifies adding or replacing models. For tuning system parameters, we construct a held-out configuration set by sampling 100 SimpleQA and 100 MMLU-Pro questions; this set is strictly disjoint from the test set in Section VI to prevent leakage.

Blending Model. OPTI-Q supports pluggable blending models for combining outputs from parallel branches. We evaluated both the GenFuser component from LLM-Blender [21] and a general-purpose LLM-based blender. In our experiments, Gemma-3:27B produced higher validation QoA than GenFuser on the held-out configuration set described above, so we use Gemma-3:27B as the designated blending model. The blender receives the original question, a blending prompt (p_{bld}), and the concatenated branch outputs, i.e., $(p_{bld} \oplus \mathrm{Par}(Q, \dots))$, and produces a single consolidated answer.

Question Processing and PERFDB Initialization. OPTI-Q initializes PERFDB with per-model and per-topic planning parameters, including model size and empirical estimates of cost, latency, energy, and QoA. These priors are populated from public benchmarking sources, including ML.ENERGY [37] and Artificial Analysis[7], and are refined as additional execution traces are collected.

At question time, OPTI-Q extracts topics for an incoming question Q using BERTopic [48]. Concretely, we embed Q with a pretrained transformer encoder, reduce dimensionality with UMAP [49], cluster with HDBSCAN [50], and derive representative topic keywords using c-TF-IDF [48]. The inferred topics are then added to the question tuple used by the planners and PERFDB lookups.

Prompt Design. We consider four prompting strategies: *Zero-Shot (ZS) [51]*, *Few-Shot (FS) [52]*, and *Chain-of-Thought (CoT) [14]*, and a *baseline* strategy (B) (i.e., just instructions without examples or reasoning). We also define two auxiliary prompts: a *context prompt* pt_{ctx} for sequential operations (ensuring the successor treats the predecessor output as evidence) and a *blending prompt* pt_{bld} for parallel operations (instructing the blender to reconcile redundant or conflicting answers). Table III summarizes the results of the prompting evaluation. *CoT* achieved the highest QoA (median 0.25, $+40$–50% over *B*), particularly on deduction-heavy domains (e.g., History, Politics), but shows the largest

[5]Code, datasets, prompts, and results are available at https://github.com/Aamir7693/Opti-Q.git.

[6]https://ollama.com

[7]https://artificialanalysis.ai/

variability (IQR $= 0.27$). *ZS* and *FS* delivered moderate, more consistent gains (median QoA 0.22 and 0.19 vs. 0.18 for *B*) with lower variability (IQR ≈ 0.23). All strategies degrade on more subjective domains (e.g., Sports, Music). Prompting also affect overheads: *FS* was the most expensive (average cost $\$0.128$, latency 136.2 s, energy 113.7 J); *ZS* was the most efficient (average $\$0.045$, 49 s, 39.5 J); *CoT* and *B* fall in between ($\sim$ $\$0.05$, $\sim$ 53 s, $\sim$ 43 J). Within OPTI-Q, these trade-offs motivate considering both CoT (higher quality, higher variance) and ZS (lower cost, higher stability). In our experiments, we use ZS since it offers the best trade-off between stability and efficiency, aligning with OPTI-Q 's emphasis on mitigating the effects of inherent LLM nondeterminism.

TABLE III: Performance, QoA, of different prompting strategies.

Prompting Strategy	**Mean**	**Median**	**Std Dev**
Few-Shot (FS)	0.23	0.19	0.19
Zero-Shot (ZS)	0.26	0.22	0.19
Chain-of-Thought (CoT)	0.30	0.25	0.21
Baseline (B)	0.22	0.18	0.20

VI. EXPERIMENTAL EVALUATION

Evaluation setup. Our evaluation targets two foundational QA paradigms using specialized benchmarks: SIMPLEQA, an open-ended factual QA dataset containing 4K questions, and MMLU-PRO, a domain-specific dataset comprising 12K multiple choice questions. Because exhaustive evaluation would entail (≈3.2M plan executions across five models), we instead build a representative, computationally tractable testbed using stratified sampling: we select 10 diverse question types from each dataset (e.g., *History*, *Mathematics*, *Sports*) and uniformly sample 10 questions per type, yielding 200 questions total (100 per benchmark) spanning factual, analytical, and reasoning-intensive tasks. Even under this stratified design, the evaluation remained computationally intensive, requiring roughly 1,000 GPU-hours overall. Experiments run on a dedicated high-performance cluster equipped with $2\times$24-core Intel Xeon Gold 6240R (2.40 GHz, 165 W TDP) CPUs and x4 NVIDIA RTX 6000 GPUs, with shared central storage exceeding 3 PB. To account for variability in LLM outputs, we execute each question five times under fixed decoding configurations and report micro-averaged QoA, latency, energy, and financial cost with 95% confidence intervals.

Planner Comparison. We compare OPTI-Q's planner backends—NSGA-II, exact DP, pruned DP, Hill Climbing, and pruned Hill Climbing—as the maximum number of operations k increases. The full comparison table is provided in For each $k \in \{1, \ldots, 5\}$, we measure (i) planning time and (ii) Pareto-front quality using hypervolume (HV) and inverted generational distance (IGD), computed relative to the exact DP frontier (when tractable). For fair comparison under matched computational budgets, the hill-climbing planner is run with a per-question timeout equal to the maximum wall-clock time observed for NSGA-II on the same question under the same operation limit. Exact DP recovers the reference frontier but exhibits state-space explosion as k grows (from near-zero planning time at small k to over 200 s at k=5). Pruned DP substantially reduces planning time (down to seconds at k=5) at the cost of lower frontier quality. Hill Climbing, under the same time budget as NSGA-II, degrades more sharply at larger k due to local optima; adding pruning further reduces frontier quality with little runtime benefit under timeout-bounded search. NSGA-II provides the best scalability–quality trade-off, maintaining near-reference frontier quality while keeping planning time in the low tens of seconds at k=5 (21 s), motivating it as the default backend for larger plan spaces. Unless noted otherwise, in subsequent experiments, we use pruned DP as the default DP-based planner and Hill Climbing with strict Pareto dominance (i.e., without the Δ-QoA filter) as the default Hill Climbing variant, with per-question timeout matched to NSGA-II for the same question and operation limit, based on the above runtime–quality trade-offs.

TABLE IV: Algorithm comparison across DAG sizes $k = 1$–5, averaged over 10 topics (3 runs for stochastic algorithms). HV% and IGD are computed w.r.t. the DP Pareto set when DP is tractable. Best values per k are in **bold** (excluding the DP reference for HV% and IGD). ↓: lower is better, ↑: higher is better.

k	Alg.	Time	HV%	IGD
1	NSGA-II	1.3 ± 0.0	$\mathbf{100.0 \pm 0.0}$	$\mathbf{0.0000 \pm 0.0000}$
	DP	**0.0**	100.0	0.0000
	DP+Δ	**0.0**	100.0	0.0000
	HC	1.3 ± 0.0	$\mathbf{100.0 \pm 0.0}$	$\mathbf{0.0000 \pm 0.0000}$
	HC+Δ	1.3 ± 0.0	100.0 ± 0.0	0.0136 ± 0.0000
2	NSGA-II	3.4 ± 0.0	$\mathbf{100.0 \pm 0.0}$	$\mathbf{0.0000 \pm 0.0000}$
	DP	**0.0**	100.0	0.0000
	DP+Δ	**0.0**	99.6	0.0953
	HC	3.4 ± 0.0	$\mathbf{100.0 \pm 0.0}$	$\mathbf{0.0000 \pm 0.0000}$
	HC+Δ	3.4 ± 0.0	99.3 ± 0.0	0.1907 ± 0.0000
3	NSGA-II	10.2 ± 0.1	$\mathbf{100.0 \pm 0.0}$	$\mathbf{0.0000 \pm 0.0000}$
	DP	**0.4**	100.0	0.0000
	DP+Δ	0.1	97.7	0.0955
	HC	10.3 ± 0.0	99.7 ± 0.3	0.0130 ± 0.0076
	HC+Δ	10.3 ± 0.0	97.3 ± 0.3	0.1690 ± 0.0334
4	NSGA-II	10.1 ± 0.1	$\mathbf{100.0 \pm 0.0}$	$\mathbf{0.0001 \pm 0.0000}$
	DP	8.0	100.0	0.0000
	DP+Δ	**0.4**	94.4	0.1191
	HC	10.2 ± 0.0	93.2 ± 1.0	0.0824 ± 0.0159
	HC+Δ	10.4 ± 0.3	91.2 ± 1.1	0.1575 ± 0.0285
5	NSGA-II	21.0 ± 0.3	$\mathbf{99.9 \pm 0.1}$	$\mathbf{0.0043 \pm 0.0016}$
	DP	208.5	100.0	0.0000
	DP+Δ	**1.9**	92.3	0.1123
	HC	21.1 ± 0.1	78.4 ± 4.4	0.1078 ± 0.0175
	HC+Δ	21.1 ± 0.1	76.9 ± 4.3	0.1352 ± 0.0247

Baselines and scope. OPTI-Q is designed for broad open-domain QA, while most existing cost-aware multi-LLM methods target classification tasks. To enable a comparison on MMLU-Pro, we treat each question as a classification and compare OPTI-Q against THRIFTLLM [31], LLM-ENSEMBLE [53], FRUGALGPT [29] and LLM-

BLENDER [21]. Because LLM-ENSEMBLE lacks a budget constraint, we implement a budgeted variant that greedily selects top-K weighted models until the budget is met [53]. For LLM-BLENDER, we adopt the latest *PairRanker* and *GenFuser* components with the author-recommended settings. On SimpleQA, we compare OPTI-Q to FRUGALGPT and LLM-BLENDER using the released configurations. Unless noted, all methods (OPTI-Q and baselines) were evaluated under identical budget constraints. We omit *LLM-TopLa* (repository errors), Shekhar et al. (summarization task), and *Palimpzest* (declarative AI analytics pipelines over document corpora) as they are not directly comparable under our QA-oriented orchestration setting.

A. Baseline Comparison

We evaluate all baseline systems under a unified cost–accuracy framework. Each method is configured to adhere to the same strict per-question budget as OPTI-Q. However, *LLM-Blender* is not budget-aware and invokes its full model set for every query, leading to different average costs across questions: $\approx 4.4\times10^{-5}$ USD/query for the *MMLU-Pro* classification benchmark and $\approx 1.6\times10^{-5}$ USD/query for the *SimpleQA* open-ended benchmark. Hence, it is shown in our figures for completeness but excluded from cost–accuracy comparisons, and compared against OPTI-Q solely on the basis of QoA. Furthermore, we adapt FrugalGPT to operate under our strict per-query budget constraint rather than its native expected-cost scheduling mechanism to align its behavior with other baselines. All baseline systems were re-implemented using our selected model suite and benchmark datasets; we denote these adapted versions with an asterisk (e.g., *FrugalGPT**, *ThriftLLM**). Hereafter, all baseline names refer to these re-implemented variants. The evaluation spans five budget levels, $b \in \{1,\ldots,5\}$, each scaling a baseline single-model usage cost of 1.2×10^{-5}. This value represents the average per-question execution cost across all candidate LLMs. Thus, the maximum allowable financial cost per question is defined as $F_{\max}(b) = b \cdot 1.2 \times 10^{-5}$.

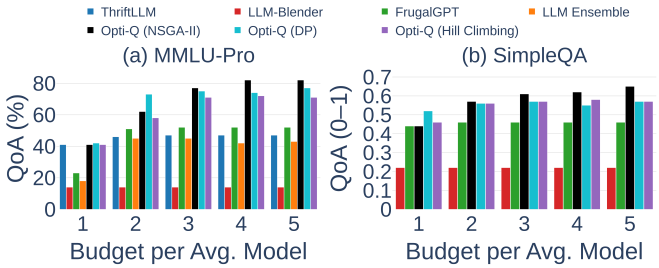


Fig. 3: QoA versus budget across two benchmarks: (a) *MMLU-Pro*, where QoA is measured as the percentage of questions answered correctly, and (b) *SimpleQA*, where QoA is measured as the average quality per question.

a) Classification-reasoning QA (MMLU-PRO): We first evaluate OPTI-Q on the *MMLU-Pro* benchmark, a multiple-choice QA task where each question has fixed answer options, and performance is measured by accuracy under varying budget constraints. At the lowest budget (b=1), as shown in Figure 3(a), *ThriftLLM** is competitive with the OPTI-Q planner variants (about 41%), due to its routing mixture between Mistral and Qwen models and its per-question ensemble aggregation, which balance cost and model capability. OPTI-Q (DP) is slightly higher at approximately 42%. As the budget increases (b=2−3), all baseline systems converge around 45–47% accuracy and subsequently plateau. This stagnation arises because these methods execute all models in parallel and deterministically collapse their predictions into a single label via confidence ranking or majority voting, thereby discarding contextual and complementary evidence among models. Consequently, additional model invocations increase the cost without improving the accuracy. In contrast, OPTI-Q improves substantially with budget. At b=2, OPTI-Q (DP) performs best ($\approx$ 73%), while at b=3, OPTI-Q (NSGA-II) overtakes ($\approx$ 77% versus $\approx$ 75% for DP and $\approx$ 71% for Hill Climbing). Unlike deterministic aggregation strategies, OPTI-Q constructs question-specific sequential/parallel/hybrid execution plans that preserve inter-model reasoning context and allocate budget adaptively across dependent and independent operations. This enables continued gains beyond the baseline plateau. Cost analysis shows that ThriftLLM* and FrugalGPT* consume, on average, 58–76% across budget levels, while LLM-Ensemble* consumes 53–99%. In contrast, OPTI-Q uses 64–88% of the available budget through selective plan allocation while achieving substantially higher accuracy. Overall, these results demonstrate that deterministic parallel ensembles exhibit early efficiency but limited scalability, whereas adaptive planning in OPTI-Q breaks this plateau by dynamically composing sequential, parallel, and hybrid plans to achieve superior cost–accuracy efficiency on classification-style QA tasks. Compared to LLM-Blender, OPTI-Q attains substantially higher QoA (0.82 vs. 0.14), underscoring the benefit of adaptive execution planning.

b) Open-Ended QA (SimpleQA): We next evaluate OPTI-Q on the *SimpleQA* benchmark (Figure 3(b)), an open-ended QA task that tests the model's ability to synthesize and articulate factual knowledge. Performance is measured by QoA, range [0,1], computed as the cosine similarity between predicted and reference embeddings. Among the baselines, FrugalGPT* remains nearly flat at around 0.46 QoA across budgets, consuming on average 58–76% of the budget (mean cost 7.2×10^{-6}–1.4×10^{-5}). LLM-Blender* (shown for completeness but excluded from cost-normalized comparisons due to fixed full-ensemble invocation) incurs a nearly fixed cost of $\approx 1.6\times10^{-5}$ USD/query and remains substantially lower at about 0.22 QoA. In contrast, all three OPTI-Q planner variants outperform these baselines. At the lowest budget (b=1), OPTI-Q (DP) performs best ($\approx$ 0.52), followed by OPTI-Q (Hill Climbing) ($\approx$ 0.46) and OPTI-Q (NSGA-II) ($\approx$ 0.44). At moderate budgets (b=2−3), the three planners converge to similar QoA ($\approx$ 0.56–0.61). At higher budgets (b=4−5), OPTI-Q (NSGA-II) achieves the best QoA, improving to about 0.65 while utilizing 64–88% of the available budget, whereas

DP and Hill Climbing plateau near 0.55–0.58.

B. Impact of Number of Operations

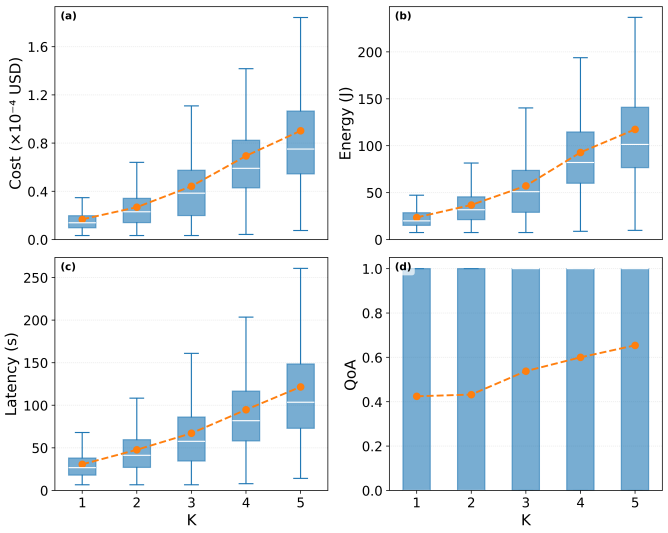


Fig. 4: NSGA-II performance across operation limits k on *MMLU-Pro*. Panels show per-question distributions of (a) financial cost, (b) energy, (c) latency, and (d) QoA. Boxplots indicate the median and interquartile range; the dashed line with markers shows the mean. Increasing k improves QoA but also increases cost, energy, and latency.

For *MMLU-Pro*, which relies on exact-match multiple-choice accuracy, the QoA distribution is effectively binary (the 0.5–0.8 bucket remains empty). Here, increasing k from 1 to 5 systematically shifts outcomes from the low-QoA bucket ($57.55\% \rightarrow 34.58\%$) to the high-QoA bucket ($42.45\% \rightarrow 65.42\%$). This demonstrates that larger plans effectively correct previous misclassifications, leading to strong, consistent improvements. In contrast, *SimpleQA* presents a more nuanced, non-monotonic trend due to its open-ended nature. After an initial QoA degradation at $k = 2$ (where the low-QoA bucket spikes to 86.36%), performance steadily improves. By $k = 5$, the low-QoA bucket shrinks to 57.23%, while the middle and high-QoA buckets expand to 27.61% and 12.86%, respectively. This indicates that while larger plans consistently enhance semantic answer quality, these gains are more gradual and highly sensitive to plan composition compared to rigid exact-match tasks. Evaluating the raw trade-offs highlights the diverging cost-benefit profiles between the two benchmarks. For SimpleQA, scaling from $k = 1$ to $k = 5$ exhibits moderate returns: a 23.6% relative QoA gain drives cost, energy, and latency up by roughly $6.5\times$, $6\times$, and $3.8\times$, respectively. Conversely, MMLU-Pro demonstrates a much stronger return on resource investment at higher k. For this dataset, a substantial 54.7% QoA improvement is achieved alongside $4\times$ to $5.3\times$ increases across the resource metrics, justifying the expanded plan space. Crucially, as k grows, the planner increasingly favors concurrent execution. Hybrid

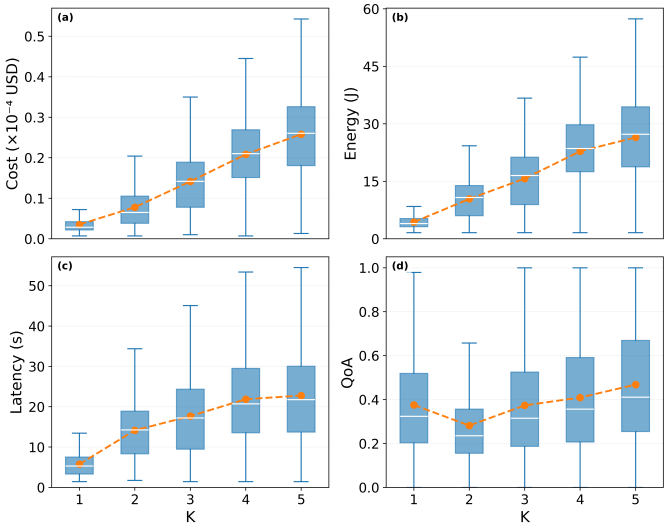


Fig. 5: NSGA-II performance across operation limits k on *SimpleQA*. Panels show per-question distributions of (a) financial cost, (b) energy, (c) latency, and (d) QoA. Boxplots summarize dispersion (median and interquartile range); the dashed line with markers shows the mean. Increasing k expands the planner's search space and improves QoA, but also raises cost, energy, and latency.

and parallel plans account for $\approx 36\%$, 77%, and 82% of all executions at $k = 3, 4$, and 5, respectively (leaving sequential plans at just $\approx 14\%$ by $k = 5$). This architectural shift explains the higher resource consumption at larger k: $k = 3$ captures most benefits for semantic QA, while classification tasks can justify larger k for higher QoA.

We next examine whether this k-dependent trend is robust across alternative planner backends. Figures 6, 7, 8 and 9 demonstrate that both DP and Hill Climbing exhibit the same qualitative pattern as NSGA-II: expanding the operations limit (k) improves QoA but predictably incurs higher financial cost, energy consumption, and latency. The primary distinction lies in the absolute QoA ceilings achieved at higher k. Hill Climbing remains highly competitive, reaching a mean QoA of 0.56 on MMLU-Pro and ≈ 0.47 on SimpleQA at $k = 5$. In contrast, DP plateaus earlier and yields lower top-end quality, peaking at ≈ 0.48 on MMLU-Pro and ≈ 0.44 on SimpleQA. The QoA-band distributions corroborate this performance gap. For example, DP on MMLU-Pro sees its high-QoA band (0.8–1.0) grow from 39.38% at $k = 1$ to 48.35% at $k = 5$, while its low-QoA band (0–0.5) drops from 60.63% to 51.65%. On SimpleQA, DP's high-QoA band only reaches 10.29% by $k = 5$. These results confirm that the fundamental trade-off—improving QoA through expanded plan spaces at the cost of diminishing efficiency returns beyond moderate k—is universal across algorithms. However, they also clearly highlight that the capacity to efficiently translate a larger search space into high-quality outcomes is highly dependent on the planner's underlying search strategy.

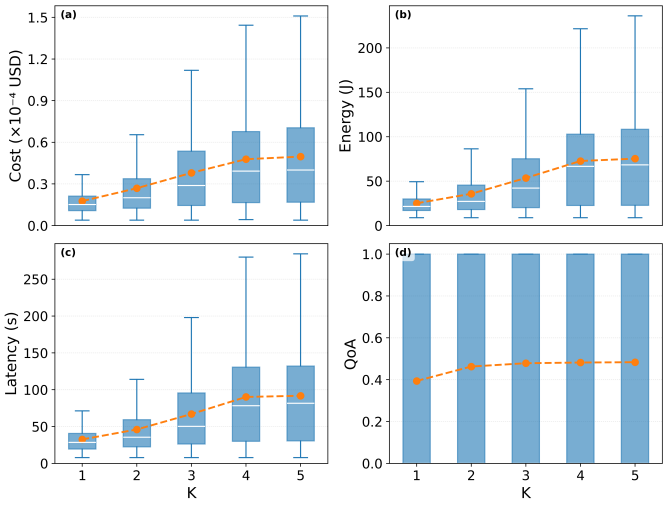


Fig. 6: DP performance across operation limits k on *MMLU-Pro*. Panels show per-question distributions of (a) financial cost, (b) energy, (c) latency, and (d) QoA. Boxplots summarize dispersion (median and interquartile range); the dashed line with markers shows the mean. Increasing k expands the planner's search space and improves QoA.

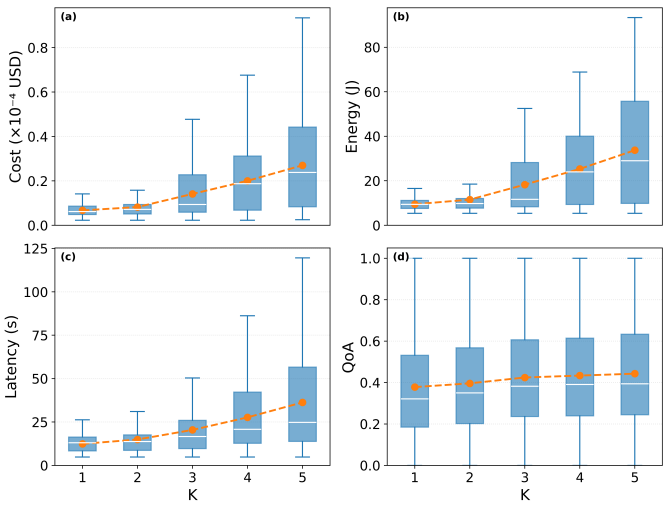


Fig. 7: DP performance across operation limits k on *SimpleQA*. Panels show per-question distributions of (a) financial cost, (b) energy, (c) latency, and (d) QoA. Boxplots summarize dispersion (median and interquartile range); the dashed line with markers shows the mean. Increasing k expands the planner's search space and improves QoA.

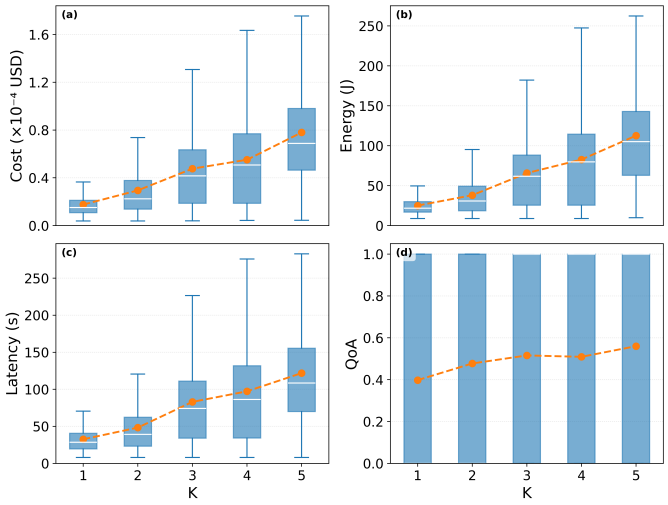


Fig. 8: Hill Climbing performance across operation limits k on *MMLU-Pro*. Panels show per-question distributions of (a) financial cost, (b) energy, (c) latency, and (d) QoA. Boxplots summarize dispersion (median and interquartile range); the dashed line with markers shows the mean. Increasing k expands the planner's search space and improves QoA.

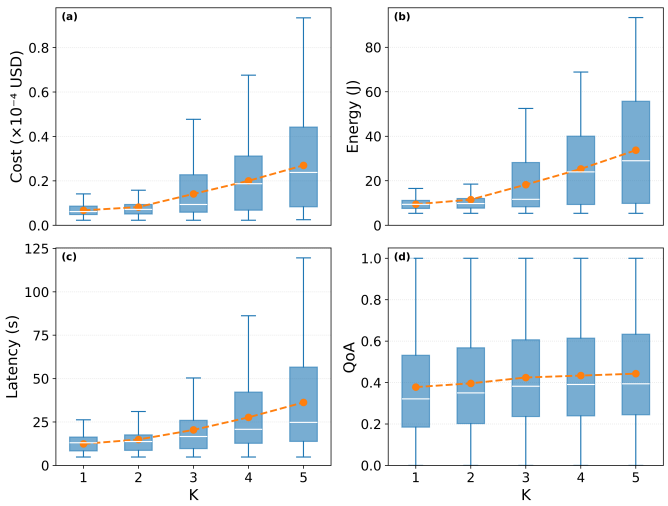


Fig. 9: Hill Climbing performance across operation limits k on *SimpleQA*. Panels show per-question distributions of (a) financial cost, (b) energy, (c) latency, and (d) QoA. Boxplots summarize dispersion (median and interquartile range); the dashed line with markers shows the mean. Increasing k expands the planner's search space and improves QoA.

C. *Robustness to Limited Historical Data in* PERFDB

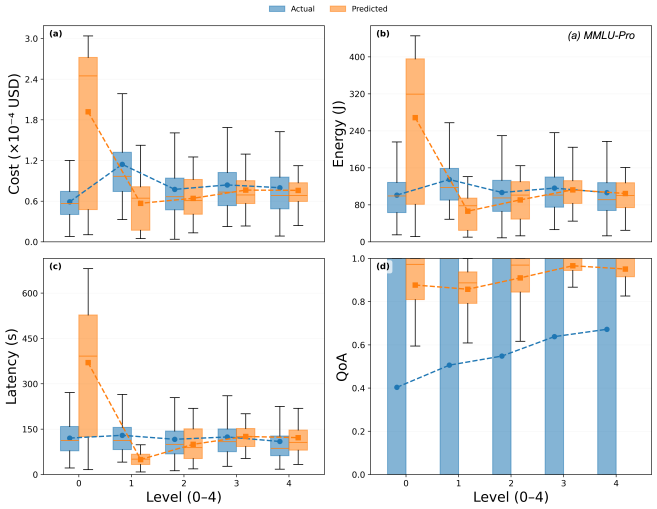
We assess how OPTI-Q behaves when PERFDB has limited historical coverage. This experiment directly studies cold-start and sparse-statistics settings by varying the amount of empirical execution metadata available to the planner. We define five coverage levels (0–4). Level 0 contains only static model priors, representing a cold-start setting with no empirical execution history. Levels 1–4 progressively enrich PERFDB with empirical traces collected from *disjoint training questions* that share the same topical distribution as the evaluation set. This setup enables the planner to generalize from related but unseen instances while avoiding data leakage. The cumulative numbers of recorded executions available in PERFDB at Levels 1–4 are 135, 985, 1,735, and 2,360, respectively. These counts denote execution traces used to populate PERFDB,

not evaluation questions. For each question, the planner can compose up to five operations (k=5), as identified in the previous experiment.

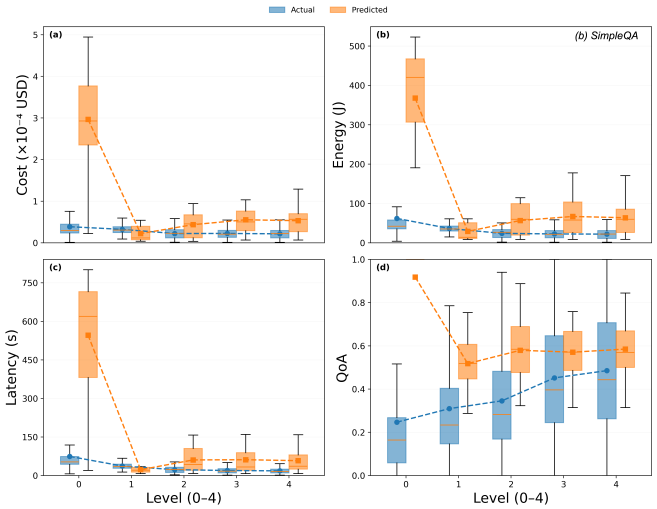
We report the full level-sweep analysis across planners, including NSGA-II, DP, and Hill Climbing, for both *MMLU-Pro* and *SimpleQA*. Across all planners, richer PERFDB coverage consistently improves planning quality and downstream QoA, with the largest gains occurring when moving from sparse to moderate coverage. For NSGA-II, Figures 10a and 10b compare predicted versus actual plan metrics across coverage levels for *MMLU-Pro* and *SimpleQA*, respectively, showing improved prediction fidelity and a shift toward higher-QoA regions as coverage increases. Table V further summarizes how the plan-QoA distribution evolves across coverage levels for each benchmark.

Richer PERFDB coverage consistently improves plan quality. For *MMLU-Pro*, QoA increases monotonically from 0.40 at Level 0 to 0.67 at Level 4 (+0.27, $\approx$ 66.7% relative improvement), while resource usage stabilizes after an early Level 1 calibration increase. Mean cost rises from 5.9×10^{-5} at Level 0 to 1.14×10^{-4} at Level 1, then settles in the 7.7×10^{-5}–8.4×10^{-5} range at Levels 2–4; energy and latency show the same pattern, peaking at Level 1 (134.8 J, 129.7 s) and then remaining broadly stable (roughly 106–116 J and 109–124 s). For *SimpleQA*, QoA increases from 0.25 to 0.48 (+0.24, $\approx$ 96.7% relative improvement), while resource usage decreases substantially: cost drops from 3.9×10^{-5} to 2.2×10^{-5}, energy from 62.1 J to 22.1 J, and latency from 74.4 s to 18.1 s (Levels 0 to 4). Thus, empirical history enables the planner to select plans that are not only higher quality but also more efficient, especially for open-ended QA where cold-start priors are conservative.

Prediction error decreases sharply as PERFDB coverage increases. For *MMLU-Pro*, MAE drops from Level 0 to Level 4 for cost ($1.37\rightarrow0.33\times10^{-4}$ USD), energy (173.8→41.5 J), latency (257.0→48.8 s), and QoA (0.58→0.34). The trend is even stronger for *SimpleQA*: cost MAE decreases from $2.58\rightarrow0.32\times10^{-4}$ USD, energy from 307.3→41.6 J, latency from 474.0→39.6 s, and QoA from 0.72→0.27. Thus, empirical traces rapidly correct cold-start bias in resource estimates, while QoA remains harder to calibrate but still improves with coverage. Table V shows that improved calibration translates into better plan selection. For *MMLU-Pro*, the high-QoA bucket (0.8–1.0) grows from 40.33% at Level 0 to 67.20% at Level 4, while the low-QoA bucket drops from 59.67% to 32.80%. The middle bucket remains empty because exact-match multiple-choice scoring produces effectively binary QoA outcomes. For *SimpleQA*, the shift is more gradual: the low-QoA bucket decreases from 87.86% to 56.00%, the middle bucket expands from 0.43% to 28.00%, and the high-QoA bucket increases from 11.71% to 16.00%. This reflects the open-ended nature of *SimpleQA*, where richer history often moves plans from poor to moderate semantic quality before reaching the highest QoA band. Overall, these results show that PERFDB provides an actionable selection signal for multi-objective planning even when absolute QoA magnitudes remain imperfect.



(a) *MMLU-Pro*



(b) *SimpleQA*

Fig. 10: Actual vs. predicted plan-level cost, energy, latency, and QoA across PERFDB coverage levels (0–4) for NSGA-II on MMLU-Pro (a) and SimpleQA (b). In both, calibration improves with historical coverage: early overestimation under sparse data (Level 0) diminishes at higher levels, while predictions preserve the level-wise trends needed for planner ranking decisions.

We next examine whether this level-dependent *calibration* trend, observed for NSGA-II above (Figures 10a and 10b), is robust across alternative planner backends. Figures 11a–11b and Figures 12a–12b compare predicted versus actual plan metrics for DP and Hill Climbing across PERFDB coverage levels. In both benchmarks, Level 0 exhibits the largest miscalibration: predicted cost/energy/latency are systematically biased relative to observed distributions, and QoA estimates are overly optimistic due to reliance on static priors. As coverage increases (Levels 1–4), the predicted distributions move closer to the observed ones and track the level-wise trends more faithfully, confirming that empirical traces improve metric

TABLE V: Plan QoA distribution (%) of solutions generated by NSGA-II across QoA buckets as a function of operation limit K and PERFDB coverage level on MMLU-Pro and SimpleQA. Larger K and higher coverage levels generally shift the distribution toward higher-QoA buckets.

Param	Val	MMLU-Pro QoA bucket (%)			SimpleQA QoA bucket (%)		
		0–0.5	0.5–0.8	0.8–1.0	0–0.5	0.5–0.8	0.8–1.0
Operation limit K							
K	1	57.55	0.00	42.45	71.63	20.00	5.12
K	2	56.79	0.00	43.21	86.36	9.63	1.87
K	3	46.22	0.00	53.78	71.74	20.22	6.01
K	4	40.00	0.00	60.00	65.22	24.51	8.48
K	5	34.58	0.00	65.42	57.23	27.61	12.86
PERFDB coverage level							
Level	0	59.67	0.00	40.33	87.86	0.43	11.71
Level	1	49.40	0.00	50.60	80.40	13.60	6.00
Level	2	45.20	0.00	54.80	76.20	17.40	6.40
Level	3	36.20	0.00	63.80	59.00	27.40	13.60
Level	4	32.80	0.00	67.20	56.00	28.00	16.00

calibration beyond NSGA-II. QoA remains the hardest metric to calibrate—especially on the open-ended *SimpleQA* task—but the prediction gap narrows with coverage and, importantly, the estimator preserves the relative ordering across levels, which is sufficient for reliable plan ranking during multi-objective search.

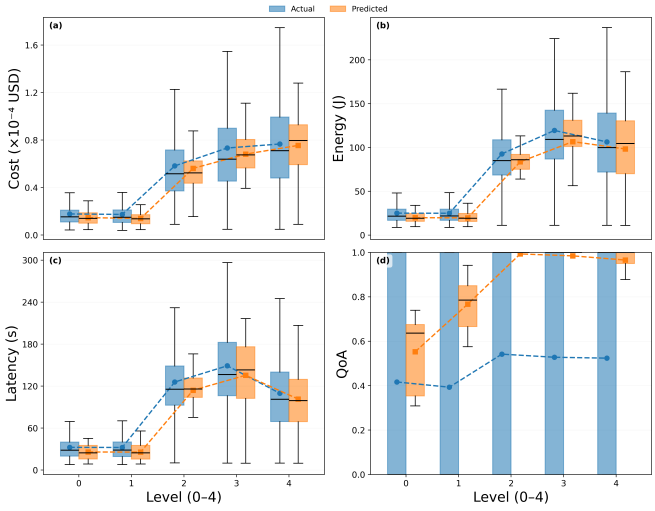
D. Impact of Model Diversity

We evaluate the effect of *LLM diversity* (i.e., the number of distinct models invoked in a plan) on QoA and resource usage. Based on the results in Section VI-B, we set the maximum number of operations K=5, with each operation allowed to call any model from the pool L). Since NSGA-II provides the best scalability–quality trade-off among the planners in our earlier comparisons, we use NSGA-II as the default planner for this experiment. We define a plan's diversity level, d, as the count of unique models invoked across all operations within it, which can range from 1 (one LLM used) to $\min(k, |L|)$ (where k is the number of operations and $|L|$ the LLM pool's size). We report micro-averaged metrics across diversity levels using *Level-4* PerfDB coverage aggregated across all budget settings, and analyze marginal efficiency ratios ΔQoA/ΔFinancial, ΔQoA/ΔEnergy and ΔQoA/ΔLatency to quantify trade-offs (see Table VI and Figure 13).

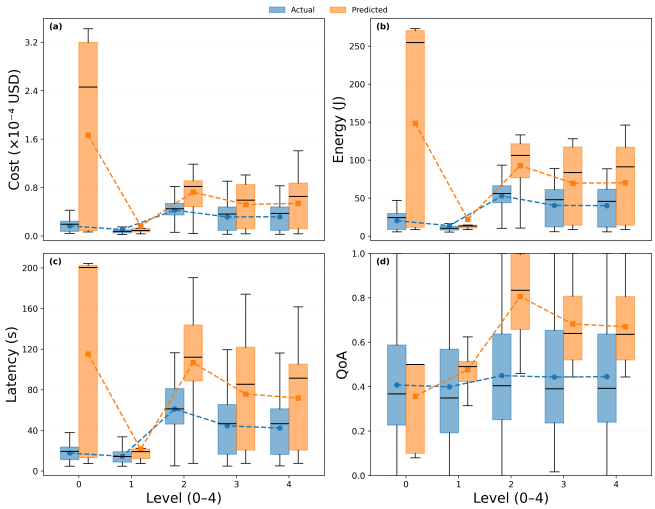
TABLE VI: Incremental changes in QoA, cost, latency, and energy for each step $d{-}1 \rightarrow d$. Cost is reported in units of 10^{-5} USD.

Metric	1→2	2→3	3→4	4→5
ΔQoA	0.021	0.038	0.019	0.008
ΔCost (10^{-5} USD)	1.20	2.60	0.85	0.25
ΔLatency (s)	17.419	22.634	-2.995	-3.732
ΔEnergy (J)	18.373	32.601	2.099	3.287
ΔQoA/ΔCost (per 10^{-4} USD)	0.175	0.146	0.223	0.314
ΔQoA/ΔLatency (per s)	0.001	0.002	-0.006	-0.002
ΔQoA/ΔEnergy (per J)	0.001	0.001	0.009	0.002

Figure 13 shows that the effect of model diversity is non-linear. QoA improves with diversity, with the largest gain from d=2→3, after which the improvements taper off. Cost and energy increase substantially up to $d = 3$ and continue to



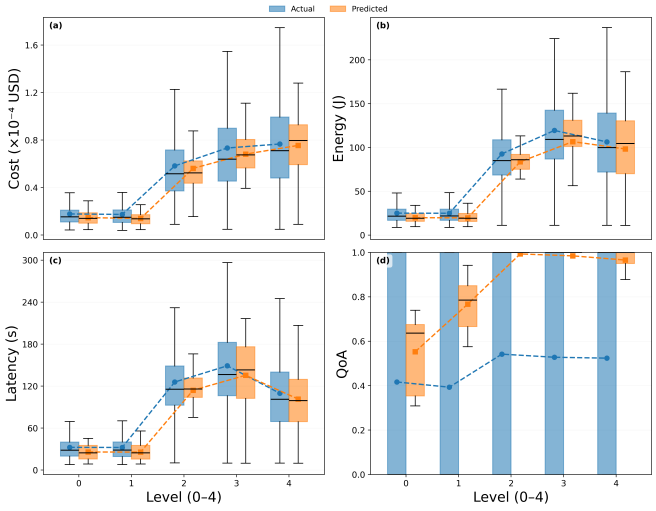
(a) *MMLU-Pro*



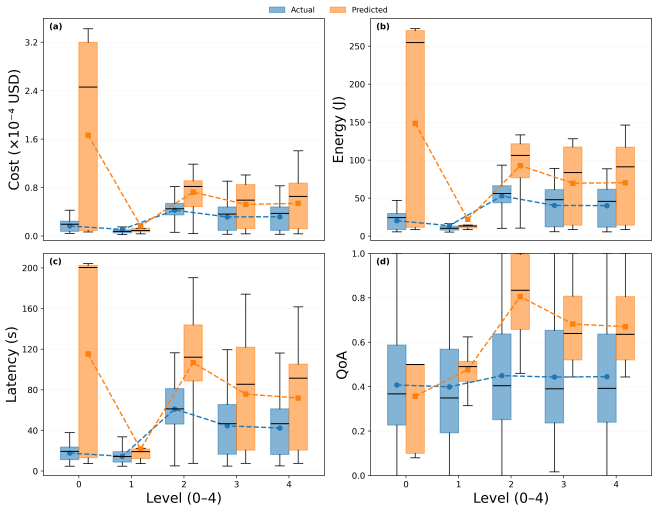
(b) *SimpleQA*

Fig. 11: Actual vs. predicted plan-level cost, energy, latency, and QoA across PERFDB coverage levels (0–4) for DP on MMLU-Pro (a) and SimpleQA (b). In both, calibration improves with historical coverage: early overestimation under sparse data (Level 0) diminishes at higher levels, while predictions preserve the level-wise trends needed for planner ranking decisions.

increase modestly for $d > 3$. Latency peaks at $d = 3$ and declines thereafter (indicating diminishing QoA returns but improved time efficiency beyond a moderate level of diversity, $d = 3$). Marginal efficiency peaks at the d=2→3 transition for latency and at the d=3→4 for energy, suggesting that $d \in 3, 4$ balances QoA and resources. Dataset effects also differ. In MMLU-Pro (knowledge-intensive), QoA generally increases with d but at higher resource cost: e.g., from $d = 1$ to $d = 5$, QoA increased in Business (0.44→0.54), Computer Science (0.30→0.64), and Physics (0.22→0.28), with Biology showing the largest gain (0.80→0.96). At the same time, for instance, in *Biology* financial cost, latency, and energy increased 3.2×, 1.2×, and 2.4×, respectively. In contrast,



(a) *MMLU-Pro*



(b) *SimpleQA*

Fig. 12: Actual vs. predicted plan-level cost, energy, latency, and QoA across PERFDB coverage levels (0–4) for Hill Climbing on MMLU-Pro (a) and SimpleQA (b). In both, calibration improves with historical coverage: early overestimation under sparse data (Level 0) diminishes at higher levels, while predictions preserve the level-wise trends needed for planner ranking decisions.

SimpleQA's QoA is flat or declines with increasing d (e.g., *Art* 0.34→0.29), with only modest gains in a few categories (e.g., *TV Shows* 0.32→0.35, *Video Games* 0.31→0.42), which are often outpaced by cost/energy growth ($> 3\times$). We next compare sequential and hybrid/parallel plans as model diversity increases. For $d \geq 2$, hybrid/parallel plans consistently achieve a higher mean QoA than purely sequential plans (e.g., at $d = 2$, 0.463 vs. 0.384), but consume more resources (higher latency and energy usage). Overall, moderate diversity ($d = 3$) is a robust operating point, achieving strong QoA with favorable marginal efficiency; increasing diversity to $d \in \{4, 5\}$ raises financial cost, latency, and energy without commensurate QoA gains.

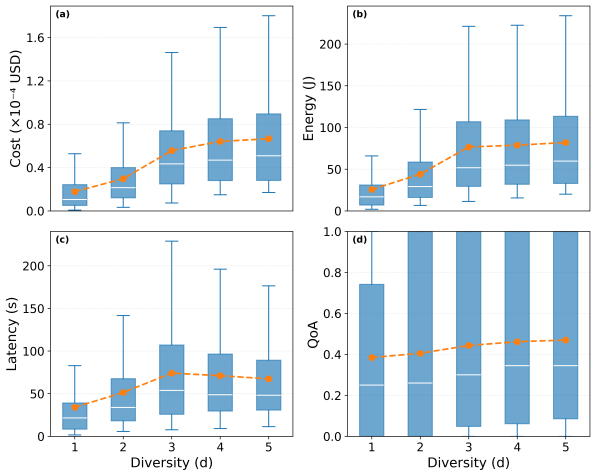


Fig. 13: Distribution of QoA across diversity levels.

E. Impact of Budget

We evaluate execution under planning-time financial and latency constraints, leaving $E_{\max}$ and $QoA_{\min}$ unconstrained[8]. A plan π is considered feasible at planning time when its estimated financial cost and latency satisfy $\text{Financial}(\pi) \leq F_{\max}$ and $\text{Latency}(\pi) \leq L_{\max}$; the planner may use up to $K{=}5$ operations. Unless noted otherwise, we use NSGA-II as the planning backend in this experiment, since it provides the best scalability–quality trade-off in our planner comparison. Budgets are instantiated at five levels using the model-averaged anchors ($\$0.000012, 19.47\,\text{s}$), and scaled as $(F_{\max}(b), L_{\max}(b)) = b \cdot (\$0.000012, 19.47\,\text{s})$ for $b \in \{1, \ldots, 5\}$. The anchor tuple specifies the mean observed financial cost and latency across the five distinct models, establishing a standardized baseline for resource allocation. Budgets are then generated by scaling this composite anchor, ensuring that each constraint is systematically derived from the aggregated typical resource usage of the system. In this setup, OPTI-Q runs NSGA-II with feasibility dominance and prunes candidates using plan-time estimates of cost and QoA. For each selected plan, we label it as *budget-adherent* if budget constraints are met after its execution and as an *overrun* otherwise. For each of the five budgets, we report the *budget-adherence breakdown* (see Table VII) as well as the QoA on budget-adherent runs (see Figure 14).

TABLE VII: Budget-adherence breakdown per budget level.

b	Budget-adherence (%)	Overrun: Fin. (%)	Overrun: Lat. (%)	Overrun: Both (%)
1	96.6	2.8	0	0.6
2	91.4	2.6	3.3	2.7
3	95.8	1.5	1.2	1.5
4	88.8	5	2.3	3.9
5	88.6	4.3	2.5	4.6

The *adherence rate* remains high across budgets (96.6% at

[8]We budget only *Financial* and *Latency*—the primary levers for fast-and-cheap interactive QA; QoA is maximized (not budgeted) and Energy closely tracks latency. Focusing on $(F_{\max}, L_{\max})$ captures the user-facing trade-off while keeping the analysis simple.

b=1 to 88.6% at b=5). Overruns are predominantly cost-driven (1.5–5.0%), with latency-only overruns at 0.0–3.3%, and joint cost+latency overruns at 0.6–4.7%. Mean QoA on budget-adherent selections shows a clear knee at b=3 (0.45), with only marginal changes thereafter (0.41 at b=4, 0.44 at b=5), indicating diminishing returns beyond moderate budgets. Dispersion increases with budget (IQR $0.31 \rightarrow 0.52$ from b=1→5), reflecting higher upside but greater variability under looser constraints. Budget utilization is conservative: budget-adherent plans consume 0.32 – 0.38 of the financial cost budget and 0.25–0.30 of the latency budget, indicating that the planner typically meets the targets without saturating them. Overall, the planner maintains high budget adherence across all levels, with overruns mainly cost-driven. QoA increases sharply at moderate budgets ($b = 3$) and then plateaus, while dispersion increases under looser budgets, indicating a trade-off between upside and variability.

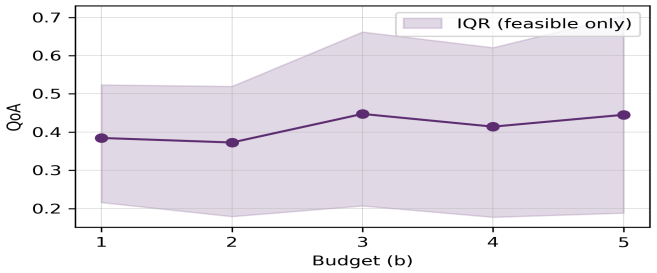


Fig. 14: QoA vs. budget (*budget-adherent plans only*).

F. Quality–Cost Trade-offs Against Commercial API Models

We evaluate whether OPTI-Q remains useful when compared against high-capability state-of-the-art commercial models accessed through their respective provider APIs (i.e., Claude Opus 4.6, GPT 5.4, and Gemini 3.5 Flash).[9] Each commercial model is invoked five times per question using the same benchmark prompt and fixed decoding configuration; we report average QoA and cost across runs. Financial cost is computed from provider-reported token usage and public pricing. We also include the best local single model per benchmark to isolate the benefit of planning over simply choosing the strongest local model. Since commercial APIs run on provider-managed backends with hidden hardware, batching, and serving policies, their latency and energy are not directly comparable to our local models served through Ollama. We therefore compare only consistently measurable metrics across both settings: QoA and financial cost. All methods use the same sampled *MMLU-Pro* and *SimpleQA* questions and the QoA metrics defined earlier.

Table VIII reports the results of the quality-cost comparison. Note that OPTI-Q provides a substantially cheaper quality–cost operating point than strong commercial API models. Local models (including the ones in OPTI-Q) where run using Ollama in our server. Hence, the cost is 0. We report in parenthesis also the cost of running the local models in an external server. Even in that case, the cost compared to commercial LLMs is minimal. W.r.t. quality, on *SimpleQA*, OPTI-Q achieves higher QoA than Claude Opus 4.6 and GPT 5.4 while costing, if executed in an external server, $37.8\times$ and $14.5\times$ less, respectively. Gemini 3.5 Flash outperforms (+0.14) but it is $95.2\times$ more expensive. On *MMLU-Pro* we observe similar performance. Hence, OPTI-Q is complementary to commercial API models. When maximum raw QoA is required, a high-capability single-call model may be preferable, but under cost-sensitive deployment, optimizer-guided local composition provides a substantially cheaper operating point with competitive (if not higher) quality.

TABLE VIII: Quality–cost comparison against strong commercial API LLM baselines.

Dataset	Method	QoA	Cost (10^{-4} USD)
SimpleQA	Claude Opus 4.6	0.597	9.59
	GPT 5.4	0.605	3.68
	Gemini 3.5 Flash	0.790	24.18
	Best local single (*LLaMA3-ChatQA (8B)*)	0.354	0 (0.071)
	OPTI-Q (NSGA-II)	0.65	0 (0.254)
MMLU-Pro	Claude Opus 4.6	0.498	18.72
	GPT 5.4	0.53	8.64
	Gemini 3.5 Flash	0.871	39.85
	Best local single (*Phi-4-14B*)	0.74	0 (0.121)
	OPTI-Q (NSGA-II)	0.82	0 (0.909)

VII. CONCLUSION AND FUTURE WORK

We presented OPTI-Q, a database-inspired optimizer for per-question multi-LLM QA planning. OPTI-Q represents LLM invocations as physical operators in a DAG, estimates QoA and resource usage using PERFDB, and searches for Pareto-efficient sequential, parallel, and hybrid plans under user-specified constraints. Across MMLU-Pro and SimpleQA, OPTI-Q achieves higher QoA at comparable cost than strong routing, cascading, and ensembling baselines, demonstrating that plan-before-execute optimization can improve quality–resource trade-offs in multi-LLM QA. These results show that structured, statistics-driven planning provides a practical foundation for adaptive LLM orchestration as model choices, budgets, and deployment backends continue to diversify.

A natural next step is to extend the same plan-before-execute abstraction beyond QA to richer RAG and agentic workflows. Because such pipelines can be expressed as DAGs over a broader operator vocabulary (e.g., retrieval, reranking, verification, and tool execution alongside generation and blending) each carrying its own quality and resource profile in PERFDB, OPTI-Q's planning machinery should transfer with little conceptual change. Realizing this will require characterizing these new operators' cost–benefit behavior and validating that the estimators remain reliable as plans grow longer and more heterogeneous, which we leave to future work.

[9] API model versions and prices were taken from provider documentation at evaluation time: Anthropic https://docs.anthropic.com/, OpenAI https://platform.openai.com/docs/models, and Google Gemini https://ai.google.dev/gemini-api/docs.

REFERENCES

[1] OpenAI, "Introducing gpt-5," https://openai.com/index/introducing-gpt-5/, 2025, accessed: 2026-02-20.

[2] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[3] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[5] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading wikipedia to answer open-domain questions," 2017. [Online]. Available: https://arxiv.org/abs/1704.00051

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[7] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[8] J. Gao, M. Galley, and L. Li, "Neural approaches to conversational ai: Question answering, task-oriented dialogues and social chatbots," 2019.

[9] T. W. Bickmore and R. W. Picard, "Establishing and maintaining long-term human-computer relationships," *ACM Trans. Comput. Hum. Interact.*, vol. 12, pp. 293–327, 2005. [Online]. Available: https://api.semanticscholar.org/CorpusID:946518

[10] A. Hamid, H. R. Samidi, T. Finin, P. Pappachan, and R. Yus, "Genaipabench: A benchmark for generative ai-based privacy assistants," *arXiv preprint arXiv:2309.05138*, vol. n/a, no. n/a, p. n/a, 2023.

[11] B. Cottier, R. Rahman, L. Fattorini, N. Maslej, T. Besiroglu, and D. Owen, "The rising costs of training frontier ai models," *arXiv preprint arXiv:2405.21015*, vol. n/a, no. n/a, p. n/a, 2024.

[12] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 3645–3650. [Online]. Available: https://aclanthology.org/P19-1355

[13] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," 2022. [Online]. Available: https://arxiv.org/abs/2204.05149

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[15] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, A. Glaese, J. Schulman, and W. Fedus, "Measuring short-form factuality in large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.04368

[16] S. Feng, W. Ding, A. Liu, Z. Wang, W. Shi, Y. Wang, Z. Shen, X. Han, H. Lang, C.-Y. Lee, T. Pfister, Y. Choi, and Y. Tsvetkov, "When one llm drools, multi-llm collaboration rules," 2025. [Online]. Available: https://arxiv.org/abs/2502.04506

[17] T. G. Dietterich, "Ensemble methods in machine learning," in *Proceedings of the First International Workshop on Multiple Classifier Systems*, ser. MCS '00. Berlin, Heidelberg: Springer-Verlag, 2000, p. 1–15.

[18] X. Amatriain, "Prompt design and engineering: Introduction and advanced methods," 2024. [Online]. Available: https://arxiv.org/abs/2401.14423

[19] S. Suzuoki and K. Hatano, "Reducing hallucinations in large language models: A consensus voting approach using mixture of experts," 2024.

[20] P. Dey, S. Merugu, and S. Kaveri, "Uncertainty-aware fusion: An ensemble framework for mitigating hallucinations in large language models," *arXiv preprint arXiv:2503.05757*, vol. n/a, no. n/a, p. n/a, 2025.

[21] D. Jiang, X. Ren, and B. Y. Lin, "Llm-blender: Ensembling large language models with pairwise ranking and generative fusion," 2023. [Online]. Available: https://arxiv.org/abs/2306.02561

[22] LangChain, *LangChain Documentation*, 2024, accessed: 2024-02-05. [Online]. Available: https://python.langchain.com/docs/introduction/

[23] C. Liu, M. Russo, M. Cafarella, L. Cao, P. B. Chen, Z. Chen, M. Franklin, T. Kraska, S. Madden, R. Shahout, and G. Vitagliano, "Palimpzest: Optimizing ai-powered analytics with declarative query processing," in *Proceedings of the Conference on Innovative Database Research (CIDR)*.

[24] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1275–1288. [Online]. Available: https://doi.org/10.1145/3448016.3452838

[25] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li, and B. Cui, "Cardinality estimation in dbms: a comprehensive benchmark evaluation," *Proc. VLDB Endow.*, vol. 15, no. 4, p. 752–765, Dec. 2021. [Online]. Available: https://doi.org/10.14778/3503585.3503586

[26] Z. Yao, H. Li, J. Zhang, C. Li, and H. Chen, "A query optimization method utilizing large language models," *arXiv preprint arXiv:2503.06902*, 2025.

[27] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou, "Lero: A learning-to-rank query optimizer," *Proc. VLDB Endow.*, vol. 16, no. 6, p. 1466–1479, Feb. 2023. [Online]. Available: https://doi.org/10.14778/3583140.3583160

[28] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "Dspy: Compiling declarative language model calls into self-improving pipelines," 2024.

[29] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[30] S. F. Tekin, F. Ilhan, T. Huang, S. Hu, and L. Liu, "Llm-topla: Efficient llm ensemble by maximising diversity," 2024. [Online]. Available: https://arxiv.org/abs/2410.03953

[31] K. Huang, Y. Shi, D. Ding, Y. Li, Y. Fei, L. Lakshmanan, and X. Xiao, "Thriftllm: On cost-effective selection of large language models for classification queries," *Proc. VLDB Endow.*, vol. 18, no. 11, p. 4410–4423, Jul. 2025. [Online]. Available: https://doi.org/10.14778/3749646.3749702

[32] S. Shekhar, T. Dubey, K. Mukherjee, A. Saxena, A. Tyagi, and N. Kotla, "Towards optimizing the costs of llm usage," 2024. [Online]. Available: https://arxiv.org/abs/2402.01742

[33] M. Saeed, N. D. Cao, and P. Papotti, "Querying large language models with sql," in *International Conference on Extending Database Technology*, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257913347

[34] M. Russo, S. Sudhir, G. Vitagliano, C. Liu, T. Kraska, S. Madden, and M. J. Cafarella, "Abacus: A cost-based optimizer for semantic operator systems," *ArXiv*, vol. abs/2505.14661, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:278768629

[35] G. Sanmartino, M. Urban, P. Papotti, and C. Binnig, "The stretto execution engine for llm-augmented data systems," 2026. [Online]. Available: https://arxiv.org/abs/2602.04430

[36] M. Durasevic, F. J. Gil-Gala, and D. Jakobović, "Does size matter? on the influence of ensemble size on constructing ensembles of dispatching rules," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, ser. GECCO '23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 559–562. [Online]. Available: https://doi.org/10.1145/3583133.3590562

[37] J.-W. Chung, J. Liu, Z. Wu, Y. Xia, and M. Chowdhury, "ML.ENERGY leaderboard," https://ml.energy/leaderboard, 2023.

[38] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[39] J. You, J. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing gpu energy consumption of dnn training," *ArXiv*, vol. abs/2208.06102, no. n/a, p. n/a, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:251554526

[40] I. Trummer and C. Koch, "Approximation schemes for many-objective query optimization," in *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, 2014, pp. 1299–1310.

[41] ——, "A fast randomized algorithm for multi-objective query optimization," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 1737–1752.

[42] G. Team, "Gemma," 2024. [Online]. Available: https://www.kaggle.com/m/3301

[43] Z. Liu, W. Ping, R. Roy, P. Xu, C. Lee, M. Shoeybi, and B. Catanzaro, "Chatqa: Surpassing gpt-4 on conversational qa and rag," *arXiv preprint arXiv:2401.10225*, vol. n/a, no. n/a, p. n/a, 2024.

[44] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[45] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann, J. R. Lee, Y. T. Lee, Y. Li, W. Liu, C. C. T. Mendes, A. Nguyen, E. Price, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, X. Wang, R. Ward, Y. Wu, D. Yu, C. Zhang, and Y. Zhang, "Phi-4 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2412.08905

[46] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023. [Online]. Available: https://arxiv.org/abs/2310.06825

[47] F. Ye, M. Yang, J. Pang, L. Wang, D. Wong, E. Yilmaz, S. Shi, and Z. Tu, "Benchmarking llms via uncertainty quantification," *Advances in Neural Information Processing Systems*, vol. 37, pp. 15 356–15 385, 2024.

[48] M. Grootendorst, "Bertopic: Neural topic modeling with a class-based tf-idf procedure," *arXiv preprint arXiv:2203.05794*, vol. n/a, no. n/a, p. n/a, 2022.

[49] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," 2020. [Online]. Available: https://arxiv.org/abs/1802.03426

[50] L. McInnes, J. Healy, S. Astels *et al.*, "hdbscan: Hierarchical density based clustering." *J. Open Source Softw.*, vol. 2, no. 11, p. 205, 2017.

[51] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022.

[52] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[53] Z. Chen, J. Li, P. Chen, Z. Li, K. Sun, Y. Luo, Q. Mao, D. Yang, H. Sun, and P. S. Yu, "Harnessing multiple large language models: A survey on llm ensemble," 2025. [Online]. Available: https://arxiv.org/abs/2502.18036

APPENDIX

A. Traversal Algorithm

The following procedure 5 describes the traversal logic used to evaluate execution plans in OPTI-Q. It performs a breadth-first traversal over the DAG representation of an orchestration plan, aggregating token-based metrics (Cost, Latency, Energy, QoA) for each node while handling both sequential and parallel blending nodes. Matched and BlendRef objects are constructed as described in PROCESSPLAN.

Algorithm 5 Traversal Procedure for Plan Evaluation

Require: DAG G, Query Type QT, *Matched*, *BlendRef*
Ensure: Estimated $(Cost, Latency, Energy, QoA)$
Object: Metrics $\rightarrow$ 4-tuple $(Cost, Latency, Energy, QoA)$
Object: ProcessingTableEntry $\rightarrow$ 2-tuple $(Node, Metrics)$
Object: QueueEntry $\rightarrow$ 3-tuple $(Node, Metrics, isStartNode)$
Object: ProcessingTable $\rightarrow$ Map $Node \mapsto$ ProcessingTableEntry
Initialize queue $Qu \leftarrow \emptyset$
Initialize map $PT \leftarrow \emptyset$
Initialize set $SeenMatchedSinks \leftarrow \emptyset$
$startNodes \leftarrow$ GETNODESINDEGREE$(0, G)$
for all $L_i \in startNodes$ **do**
    $M \leftarrow (0, 0, 0, 0)$
    $isStartNode \leftarrow$ **True**
    $entry \leftarrow (L_i, M, isStartNode)$
    Qu.enqueue$(entry)$
end for
while Qu is not empty **do** ▷ Breadth-first traversal
    $(node, M, isStartNode) \leftarrow Qu$.dequeue()
    if ISBLENDNODE$(node, G)$ **AND** $node \notin$ Matched **AND** $PT[node]$.length = Indegree$(node, G)$ **then**
        $M \leftarrow$ GETPARALLELMETRICS$(node, BlendRef, PT)$
    else
        Qu.enqueue$((node, M, isStartNode))$
        continue
    end if
    $successors \leftarrow$ GETSUCCESSORS$(G, node)$
    if $|successors| = 0$ **then**
        return M
    end if
    for all $s \in successors$ **do**
        if $s \in SeenMatchedSinks$ **then**
            continue
        end if
        if $s \in Matched$ **then**
            $M \leftarrow Matched[s]$
            $entry \leftarrow (s, M,$ **False**$)$
            Qu.enqueue$(entry)$
            continue
        end if
        $(M, QueueUp) \leftarrow$ PROCESSNODE(G, QT, c, s, M, PT)
        if $QueueUp =$ **True then**
            Qu.enqueue$((s, M,$ **False**$))$
        end if
    end for
end while
return M

We employ a modified BFS traversal to estimate the Cost, Energy, Latency, and QoA of a plan. ProcessPlan A will return two mappings *Matched* and *BlendRef*, which respectively contain information pertaining to matched sub-plans and QoA information of the plan's parallel executions. The traversal will make use of these mappings to guide the estimation. There are certain Objects defined in the algorithm for the purposes of grouping essential stored information during traversal.

1) **Metrics**
2) **ProcessingTable**
3) **ProcessingTableEntry**
4) **QueueEntry**

Metrics is 4-Tuple containing the aggregated Cost, Latency, Energy, and estimated QoA during the BFS-traversal. For example at node L_i, there will be an aggregated amount for each metric stored in a **Metrics** object. When the traversal adds node L_j, a successor of L_i, to the BFS queue, during processing of L_j the aggregated metrics will be updated due to the influence of a L_j execution.

ProcessingTable is a map which maps Nodes to **ProcessingTableEntry** structures. A **ProcessingTableEntry** object is a 2-Tuple containing a node and a **Metrics** object. The purpose of ProcessingTableEntry is to store the aggregated metrics from all incoming branches to a blend node. With this, we can store all the information needed for parallel execution aggregation in an ordered fashion. In addition, we can track whether all necessary information has propagated to the blend node. For instance, say node L_i is a blend node with nodes L_1, L_2, L_3, L_4 as its inputs. To properly estimate QoA and aggregate Cost, Latency, Energy, we need to wait for simulated execution of L_1, L_2, L_3, L_4 to finish. Hence, the **ProcessingTable** and its entries will indicate when it feasible to calculate an updated set of metrics simply by checking for the existence of an entry for the input nodes.

QueueEntry is a 3-Tuple containing a node, **Metrics** object, and a boolean flag indicating whether the entry is for a start node. For the traversal function, whether a node is a start node is an important distinction because it will affect how we propagate information.

The traversal will begin by entering all start nodes of the plan into the Queue as a **QueueEntry** object, which will initially contain 0 for all metrics in the associated **Metrics** object. We also initialize the **ProcessingTable** and a set to store seen nodes. Because of the DAG structure nodes with in-degree of 1 will only be vistited at most one time. For these types of node, we do not want to re-enter them into the queue, hence the set *SeenMatchedSinks*.

Once the inital state of the queue is initialized, processing of the entry is done as such. First, we pop a *QueueEntry* from the beginning of the queue, extracting the node, accumulated **Metrics**, and isStartNode flag.

Before performing BFS on the successors, we check if the current node is a blend node and is not in *Matched*. If that criteria is met, we are dealing with a parallel execution that is also not apart of a matched subplan. In this case, we need to aggregate all the metrics of each incoming branch, and estimate the QoA with the parallel QoA estimation method. For this, the information will be contained in the processing table. However, we only aggregate if information for incoming branches in the table for the blend node. Eventually in the midst of traversal, the information will propagate. However, if we reach the blend node before this occurs, then we simply readd the blend node back into the queue, and, then, continue to the next iteration. Readding back into the queue, will allow

for reprocessing at a later time when the information will be available.

From here we proceed to analyzing the successors of the node in relation to its place in the DAG representation of the plan. Before processing each successor, we first check if the successor is the sink of a matched subplans contained in *Matched*. If so, we forget any accumulated Metrics aggregated at this point in the traversal, and, instead, and replace it with the metrics contained in *Matched* as these are metrics gathered from real-time fuzzy/exact executions. Then, for each successor a sub-procedure, ProcessNode, is called to determine how to aggregate the metrics and estimate the QoA with the current and successor Node. Because, the current and successor node can either be a blend or sequential node, there are rules followed for aggregation based on the node distinction. ProcessNode will aggregate Cost, Latency, and Energy and estimate QoA based on the type of nodes the current and successor are, sequential or blend. ProcessNode will then return the newly aggregated metrics and a boolean indicator indicating whether the successor node needs to be readded into the queue.